VAXTRACK

FUNCTIONAL SPECIFICATION DOCUMENT

VaxTrack Vaccination Management Platform

A comprehensive specification of business requirements, functional behaviour, roles, workflows, and system diagrams for the VaxTrack two-dose vaccination tracking platform (Backend v2.0.0).

DOCUMENT VERSION	2.0.0
RELEASE DATE	08 July 2026
STATUS	Draft — Pending Stakeholder Review
CLASSIFICATION	Confidential — Internal Use
PREPARED BY	VaxTrack Engineering

This document is prepared for stakeholder review and approval. Contents are subject to change following review feedback.

Document Control

Revision History

VERSION	DATE	AUTHOR	DESCRIPTION
1.0.0	2026-06-26	VaxTrack Engineering	Initial functional specification for the v2.0.0 backend (5 modules: Auth, User, Hospital, Booking, UserRoleMapping).
2.0.0	2026-07-08	VaxTrack Engineering	Full rewrite to formal FSD standard. Adds the Notification module, hospital/user lifecycle management (disable-reactivate-unregister), hospital-admin self-service applications, vaccination certificates, and audit trails. Adds business context, HLD/LLD/use-case/sequence/state diagrams, and a consolidated business rules and error-handling reference.

Distribution List

NAME / GROUP	ROLE	PURPOSE OF REVIEW
Project Sponsor / Business Stakeholder	Approver	Confirm business requirements and objectives are correctly captured.
Product / Delivery Lead	Reviewer	Validate scope, roles, and workflow completeness.
Engineering Team	Reviewer	Confirm functional behaviour matches implemented system.
QA / Test Lead	Reviewer	Use as the basis for test case design.

Related Documents

DOCUMENT	RELATIONSHIP
VaxTrack — Architecture & Operational Flows Document	Companion document — implementation-level component design and step-by-step operational traces (to be issued separately).
VaxTrack — Technical Specification Document	Companion document — technology stack, configuration, and engineering conventions (to be issued separately).
VaxTrack — Test Cases Document	Companion document — test case catalogue derived from

	this specification (to be issued separately).
VaxTrack API Contracts (Bruno collections)	Machine-usable request/response contracts for every endpoint referenced in this document.

How to read this document

Section 5 (Functional Requirements) is organised by module, and each module follows the same pattern: a short overview, a High-Level Design (HLD) diagram, a Low-Level Design (LLD) decision-flow diagram, one or more UML sequence diagrams for its key flows, a state diagram where the module owns a lifecycle, and a requirement card for every endpoint the module exposes. Business rules are consolidated in Section 7 and cross-referenced by ID (BR-xx) throughout.

Table of Contents

1. Introduction

1.1 Purpose of the Document

This Functional Specification Document (FSD) defines, in full, the business and functional behaviour of the VaxTrack vaccination management platform, Backend v2.0.0. It describes what the system does, for whom, under what rules, and with what boundaries — independent of how it is implemented. It is the reference stakeholders use to confirm that the platform, as specified, meets the business need, and the reference the engineering and QA teams use to build and verify the system consistently.

1.2 Scope of this Document

This document covers every functional module currently exposed by the VaxTrack v2.0.0 API: Authentication & Account Access, User Management, Hospital Management, Booking & Vaccination Lifecycle, Role & Access Requests, and Notifications. For each module it specifies the actors involved, the triggering conditions, the request/response behaviour, the governing business rules, and the error scenarios. It also defines the role and permission model that spans all modules, and consolidates cross-cutting concerns (business rules, error handling, assumptions, and constraints) that would otherwise be duplicated per module.

This document intentionally excludes internal implementation detail — class names, database schema, API payload field-by-field wire format, and infrastructure configuration. Those are the subject of the companion *Architecture & Operational Flows* and *Technical Specification* documents referenced in Document Control.

1.3 Intended Audience

- **Business stakeholders / sponsors** — to confirm the platform satisfies the intended business outcomes before further investment.
- **Product and delivery leads** — to validate scope, sequencing, and completeness of the feature set.
- **Engineering team** — as the behavioural contract to build against and to check implementation drift against.
- **QA / test team** — as the source from which test cases and acceptance criteria are derived.
- **Support / operations staff** — to understand what the platform is supposed to do when triaging a reported issue.

1.4 Definitions, Acronyms & Abbreviations

Term	Definition
Dose 1 / Dose 2	The first and second administrations in the platform's two-dose vaccination model. Each has its own hospital, requested date, slot, and completion state.
Booking	A single record tracking one beneficiary's Dose 1 and Dose 2 journey, from request through completion or cancellation/rejection.
Platform Admin	A user with global administrative privileges across the entire platform (referred to in code as the `admin` role).
Hospital Admin	A user granted elevated privileges scoped to exactly one hospital, via a role assignment rather than a platform-wide flag.
Scoped Role	A role that only grants elevated access within a specific context (e.g. one hospital), as opposed to a platform-wide role.
Slot	One unit of vaccination capacity at a hospital for a given day; a hospital exposes a total capacity and a live available count.
Soft Delete	A deletion pattern where a record is flagged inactive/removed rather than physically erased, preserving history and audit trail.
JWT	JSON Web Token — the bearer credential issued at login and required (unless noted "AllowAnonymous") on every subsequent request.
Audit Trail	A permanent, append-only log of who did what to a booking, hospital, or user, and when.
FR / BR	Functional Requirement / Business Rule — the numbered identifiers used throughout this document (e.g. FR-BOOK-03, BR-07).

1.5 References

- VaxTrack API Contracts — Bruno-compatible request collections, one per module, covering every endpoint referenced in Section 5.
- VaxTrack Seed Data & SQL Reference Scripts (Documents/v2_docs/vaxtrack - sql queries).
- VaxTrack v2.0.0 source repository (v2_backend/v2.0.0, v2_frontend/v2.0.0).

◎ 2. Business Overview

2.1 Business Context & Problem Statement

Vaccination programs that span many independently-operated hospitals struggle with three recurring problems: beneficiaries have no single place to book and track a two-dose course; hospitals have no shared, live view of their own slot capacity or of who is due to arrive; and program administrators have no consolidated way to oversee hospitals and enforce accountability (who approved what, who disabled which account, and why). VaxTrack addresses this by acting as the single system of record for the end-to-end vaccination journey — from registration, through slot booking and dose approval at a hospital, to a verifiable completion certificate — while giving each hospital self-service control over its own slot capacity and each level of oversight (hospital vs. platform) a clearly scoped set of permissions.

2.2 Business Objectives

- **Give every beneficiary one place** to register, discover hospitals, book both doses, and produce a verifiable certificate once vaccinated.
- **Give every hospital self-service control** of its own contact details and slot capacity, without needing platform-admin involvement for routine changes.
- **Give the platform operator full oversight** of hospitals and users — including the ability to disable, reactivate, or permanently unregister a hospital or user account, with a mandatory reason and a full audit trail.
- **Keep every state-changing action accountable**: who did it, to what, when, and (where relevant) why — surfaced through per-entity audit trails and in-app notifications.
- **Prevent capacity and identity abuse**: no double-booking, no booking on someone else's behalf, no login for a disabled account, and no permanent removal of a hospital without a second-party confirmation.

2.3 Stakeholders

Stakeholder	Interest in the System
Beneficiary (Normal User)	Wants a simple, trustworthy way to book both doses and prove vaccination status afterward.
Hospital Admin	Wants control over their own hospital's capacity and the ability to approve/reject bookings without waiting on the platform operator.
Platform Admin	Wants full oversight of hospitals and users, the ability to intervene when something goes wrong, and a defensible audit trail.
Program Sponsor / Business Owner	Wants assurance the platform enforces the program's rules (one active booking per person, dose ordering, capacity limits) without manual policing.

2.4 Success Criteria

- A beneficiary can complete registration through certificate download without any manual, off-platform intervention.
- A hospital admin can manage their hospital's slot capacity and dose approvals entirely within their own scope, with zero visibility into or effect on other hospitals.
- Every disable, reactivation, unregister, promotion, and role change is traceable to an actor, a timestamp, and (where applicable) a reason — with no gaps.
- No booking can ever push a hospital's available slots negative or above its total capacity.
- A disabled account cannot authenticate under any circumstance until explicitly reactivated by a platform admin.

3. System Overview

3.1 Product Perspective

VaxTrack is a standalone vaccination-management API consumed by a web frontend (and, by design, any Bruno/Postman-style API client or future mobile client, since every rule lives server-side). It is organised into six functional modules — Authentication & Account Access, User Management, Hospital Management, Booking & Vaccination Lifecycle, Role & Access Requests, and Notifications — each exposed as its own controller and each detailed in Section 5. The system has no external system dependencies beyond its own SQL Server database; email delivery is stubbed for a future integration (see Section 9, Assumptions).

3.2 System Architecture Summary

The backend follows a three-layer architecture — Controllers (HTTP + auth enforcement) → Services (business rules, ownership and role checks) → Repositories (data access) — sitting on top of a single SQL Server database. Full component-level and database-schema diagrams are maintained in the companion Architecture & Operational Flows document; this specification embeds only the High-Level Design (HLD) view for each module, immediately before that module's functional requirements in Section 5.

3.3 End-to-End Flow

Figure 1 traces the full lifecycle the platform supports end to end: registration and login (including the disabled-account block and reactivation path), the normal-user booking journey from Dose 1 through certificate download, the hospital-admin application and oversight path, and the platform-admin oversight functions — all tied together by the cross-cutting notification service that fires on every approval, rejection, and status change.

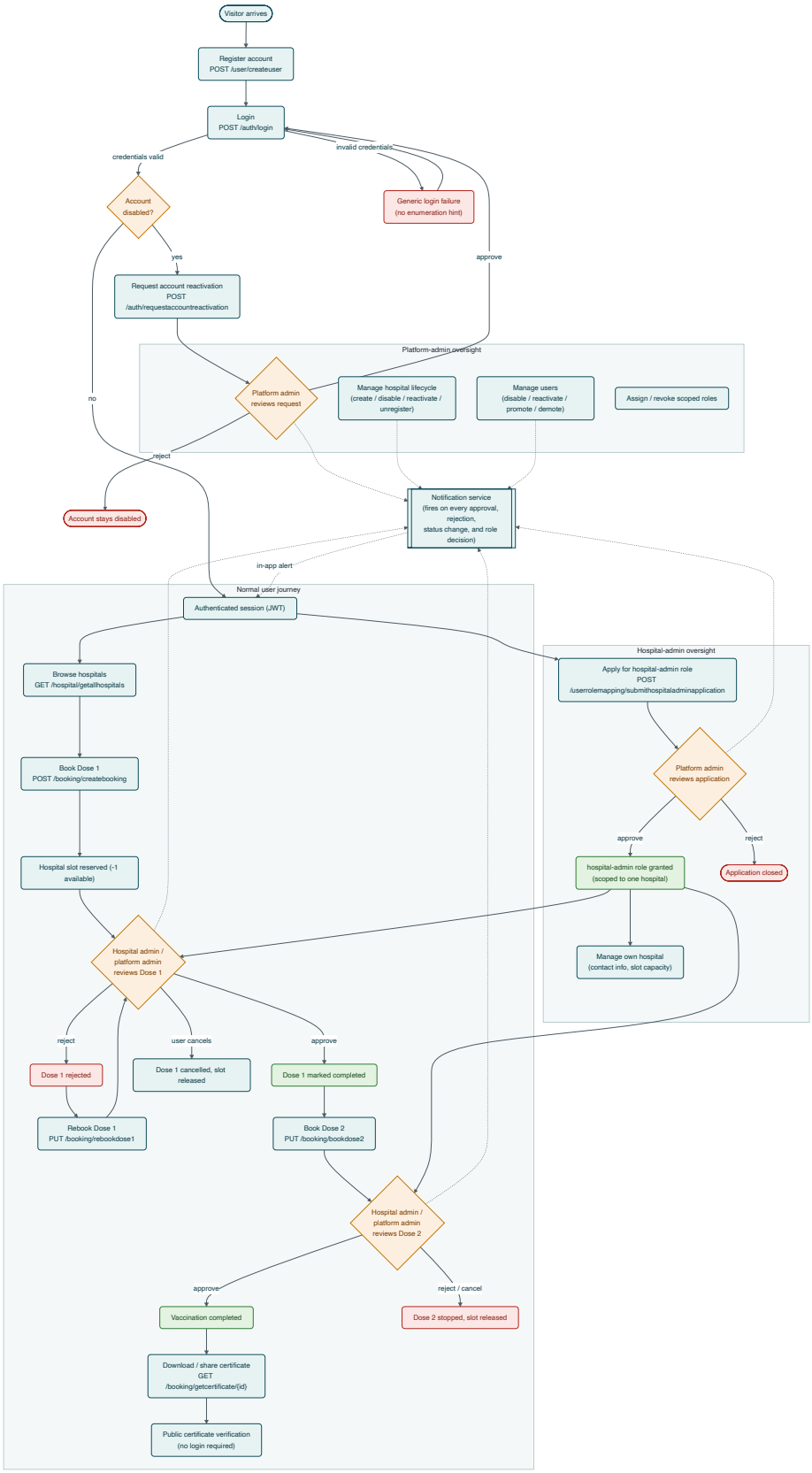

Figure 1. *End-to-end functional flow — registration through certificate, with hospital-admin and platform-admin oversight paths.*

3.4 Technology Snapshot

Provided here for orientation only — full detail is in the companion Technical Specification document.

Layer	Technology
Runtime & Framework	.NET 10 / ASP.NET Core Web API
Data Access	Entity Framework Core 10 over Microsoft SQL Server
Authentication	JWT Bearer (HMAC-SHA256), server-side revocation list for logout
Password Security	BCrypt hashing — plain text is never persisted
Frontend	Angular 20 (standalone components) with TailwindCSS

4. User Roles & Access Model

4.1 Role Definitions

Role	Definition
Normal User	The default role granted on registration. Full control over their own profile and their own booking; no visibility into other users' data.
Hospital Admin	A normal user additionally granted the `hospital-admin` scoped role for exactly one hospital (via direct admin assignment or an approved self-service application). Can manage that hospital's contact details, slot capacity, and dose approvals — and nothing outside that scope.
Platform Admin	A user with the platform-wide `admin` flag. Unconditional access across every module: user and hospital lifecycle management, all bookings, all role assignments, and every pending request queue.

4.2 Role Assignment Mechanism

The system uses two independent, layered role mechanisms:

- **Platform role** — a boolean flag on the user's account, baked into the JWT at login as the `admin` claim. Checked declaratively at the endpoint level (`[Authorize(Roles = "admin")]`).
- **Scoped role** — a row in the role-mapping table keyed by user, role tag, and a context id (a Hospital Id for `hospital-admin`). Checked at runtime, inside the service layer, against the specific entity being acted on — so a hospital admin for Hospital A is transparently rejected when attempting to act on Hospital B.

A platform admin unconditionally satisfies any scoped-role check without a database lookup. A user may hold the `hospital-admin` scoped role for more than one hospital simultaneously, via multiple independent role-mapping rows.

4.3 Roles & Permissions Matrix

Capability	Normal User	Hospital Admin	Platform Admin
Register, login/logout, forgot/reset password	Y	Y	Y
Manage own profile, email, password	Y	Y	Y
Delete own account	Y	Y	Y
Book / edit / rebook / cancel own doses	Y	Y	Y
View/download own certificate	Y	Y	Y
Apply for hospital-admin role	Y	Y	—
View all hospitals / a hospital's own profile	Y	Y	Y
Update hospital contact info / slot capacity	N	Own hosp.	Y
Approve / reject a dose	N	Own hosp.	Y
View bookings for a hospital	Y	Y	Y
View all bookings platform-wide	N	N	Y
Admin-override a booking (all fields) / delete a booking	N	N	Y
Disable / reactivate / unregister a hospital	N	Request only*	Y
Create a hospital	N	N	Y
View / manage all users, promote / demote admin	N	N	Y
Disable / reactivate a user account	N	N	Y
Assign / revoke scoped roles, review pending requests	N	N	Y

* A hospital admin may request reactivation, decline an unregister request, or authorize a pending unregister for their own hospital — but cannot initiate a disable or unregister; only a platform admin can.

4.4 Use Case Diagrams

The following four diagrams present the system's use cases grouped by functional package (a standard UML practice when the combined actor/use-case count is too large for a single legible diagram), rather than as one combined diagram.

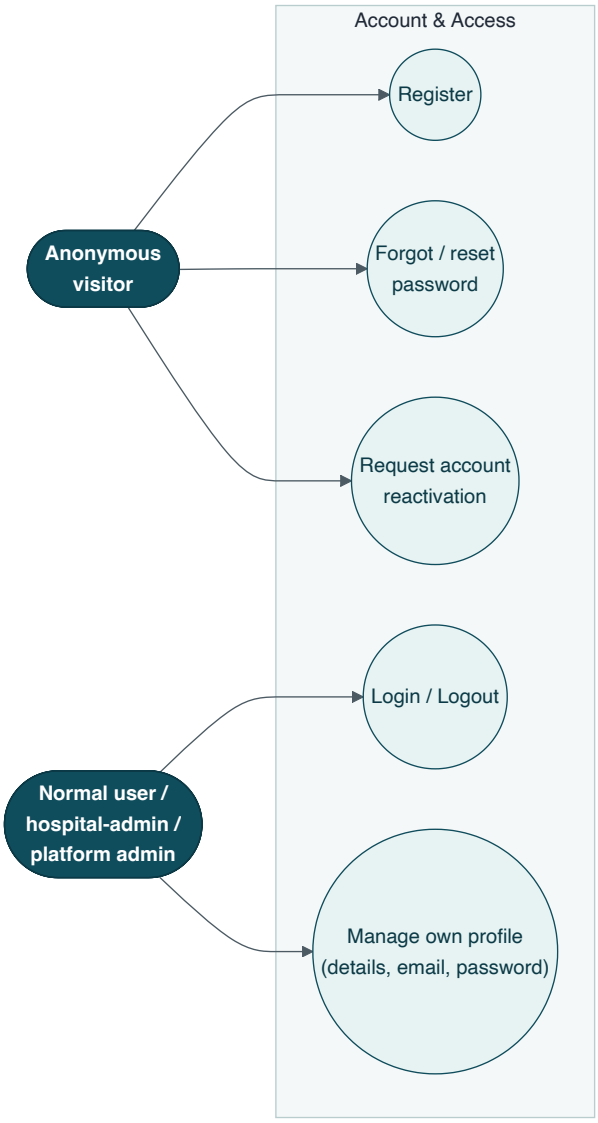

Figure 2. *Use cases — Account & Access.*

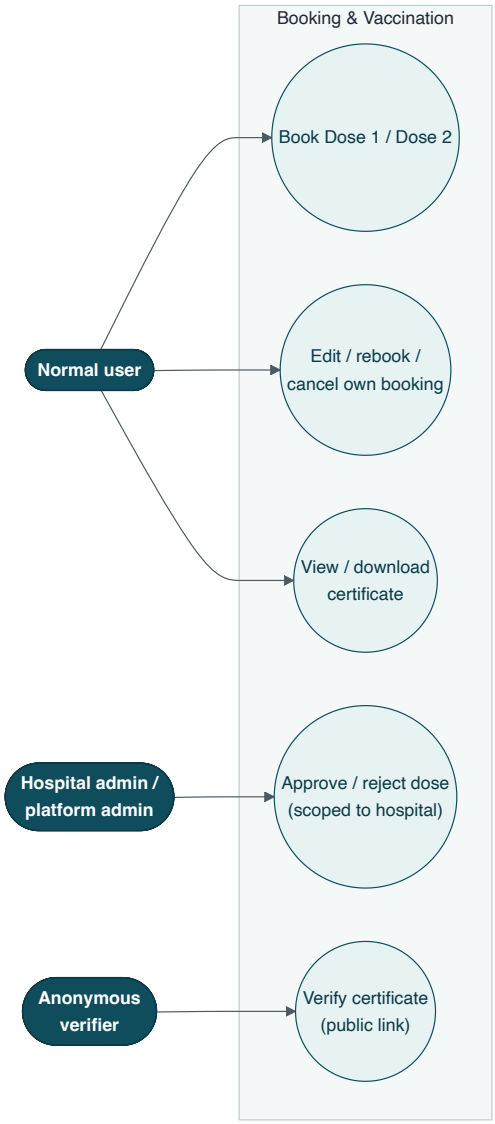

Figure 3. *Use cases — Booking & Vaccination.*

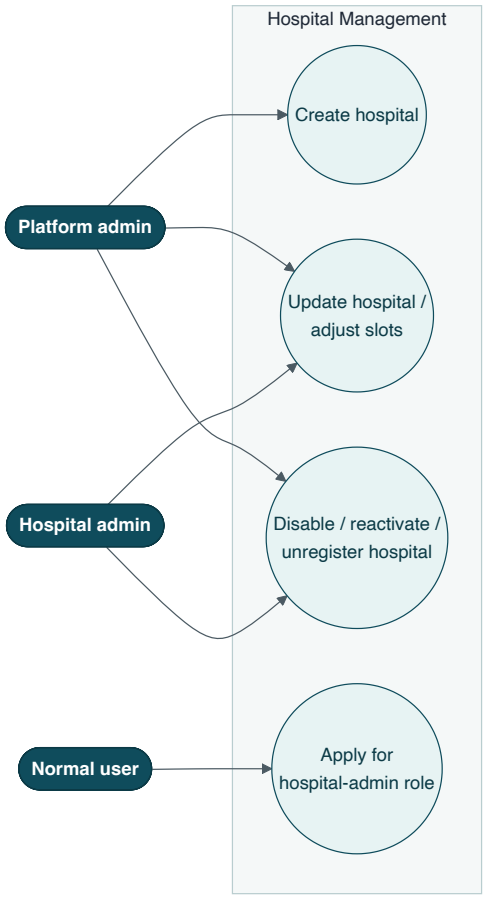

Figure 4. *Use cases — Hospital Management.*

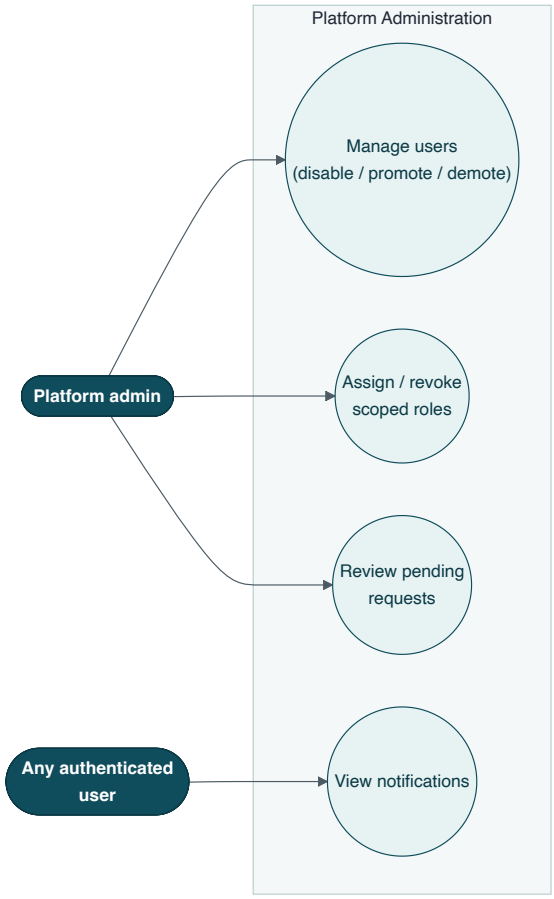

Figure 5. *Use cases — Platform Administration.*

5. Functional Requirements

This section specifies every endpoint the platform exposes, grouped into the six functional modules introduced in Section 3.1. Each module opens with an overview, its High-Level Design (component) diagram, a Low-Level Design (decision-flow) diagram, the UML sequence diagrams for its key flows, a state diagram where the module owns an entity lifecycle, and one requirement card per endpoint.

5.1 Module: Authentication & Account Access

Auth

Login, logout, password recovery, and the disabled-account reactivation path

This module is the single entry and exit point for authentication. It issues and revokes JWTs, and owns the two self-service recovery paths a user needs when they cannot log in: a forgotten password, and a disabled account. It deliberately returns identical, generic responses for any credential failure (wrong password, unknown email, or a soft-deleted account) to prevent account enumeration — see BR-19.

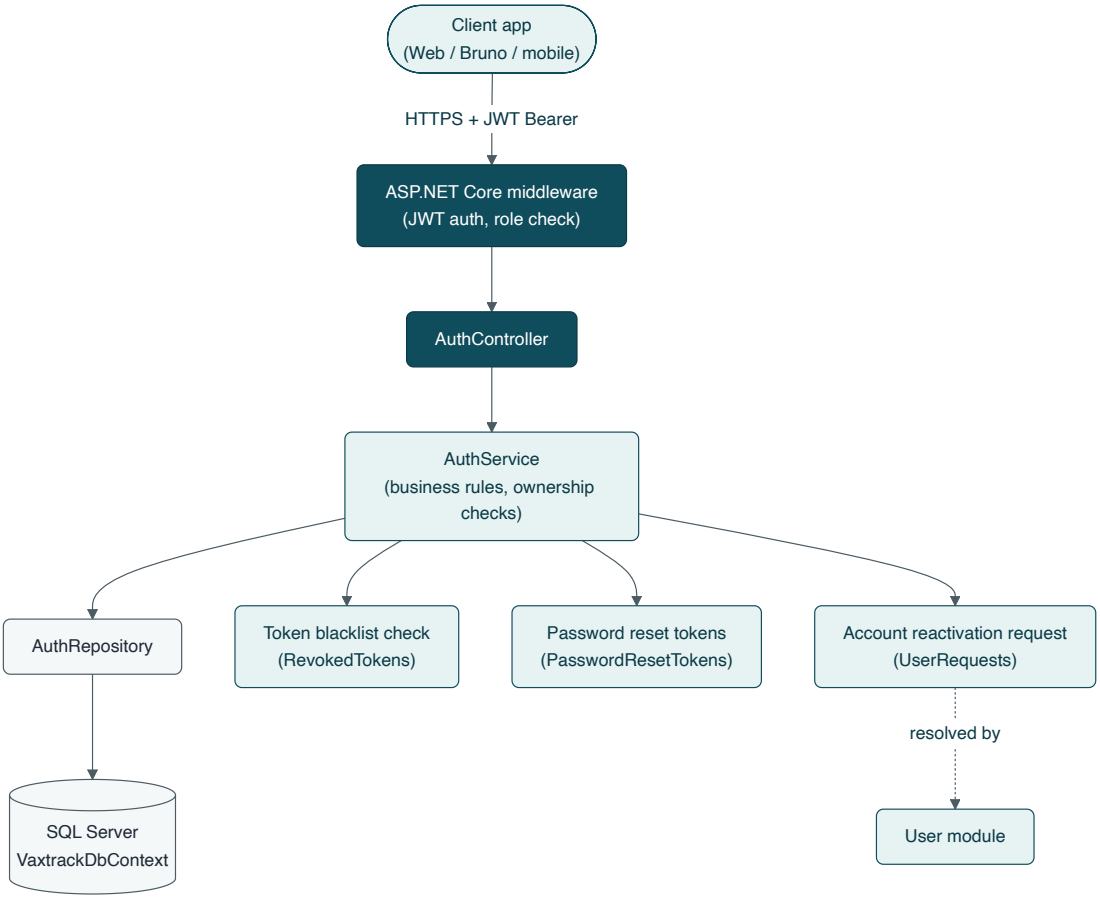

Figure 6. *High-Level Design — Auth module.*

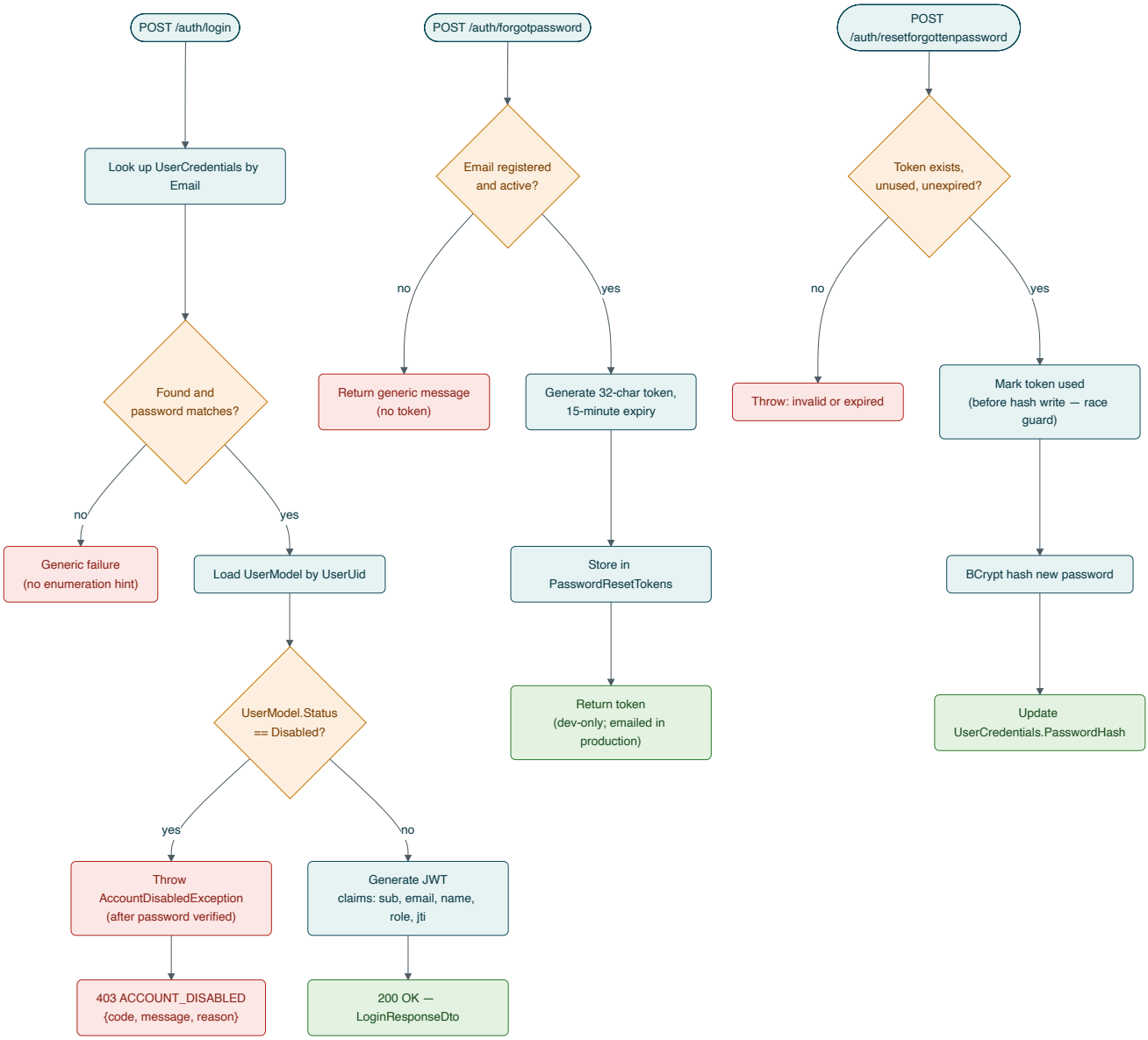

Figure 7. *Low-Level Design — login and password-recovery decision flow.*

Key Flows

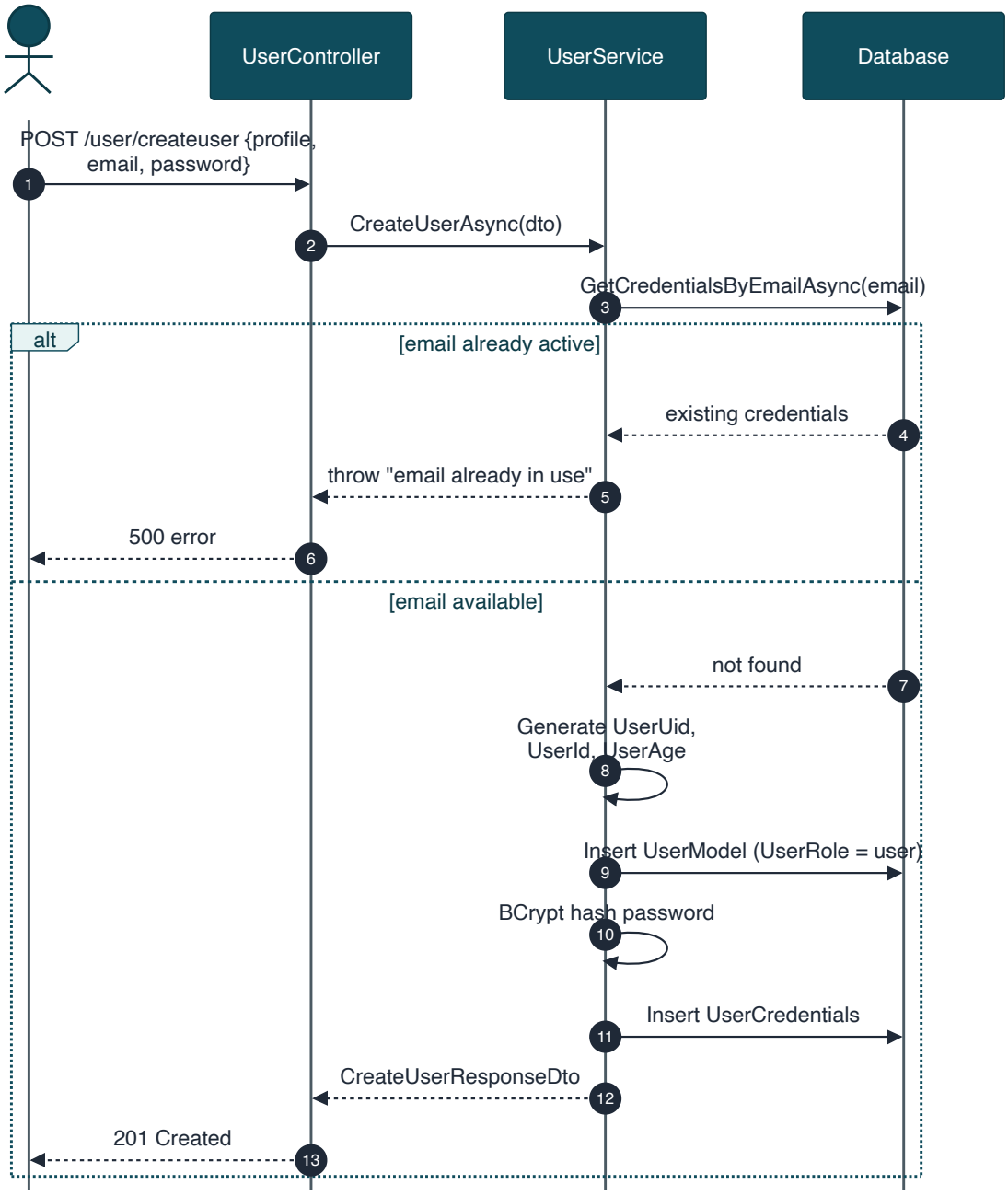

Figure 8. *Sequence — user registration.*

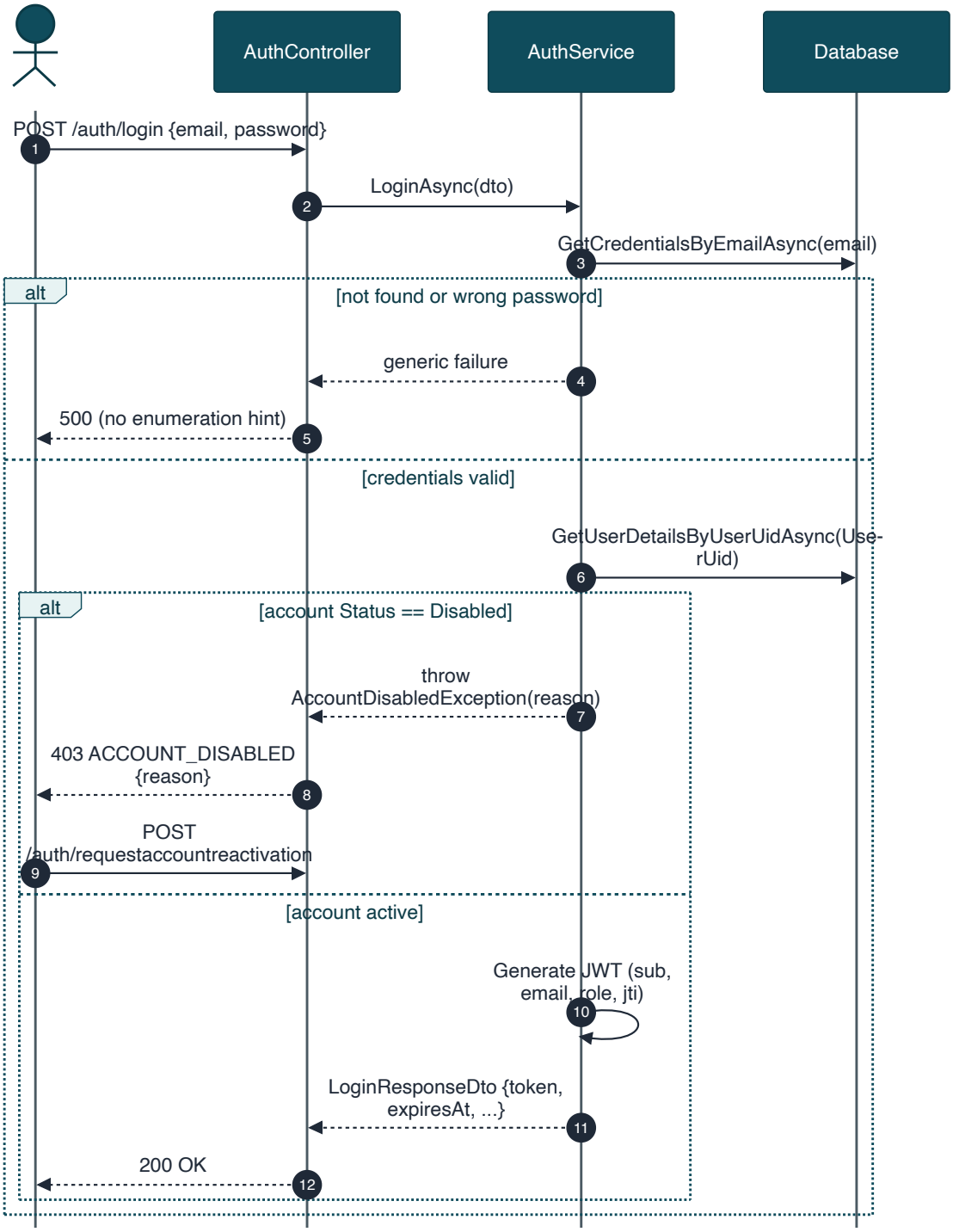

Figure 9. *Sequence — login, including the disabled-account block.*

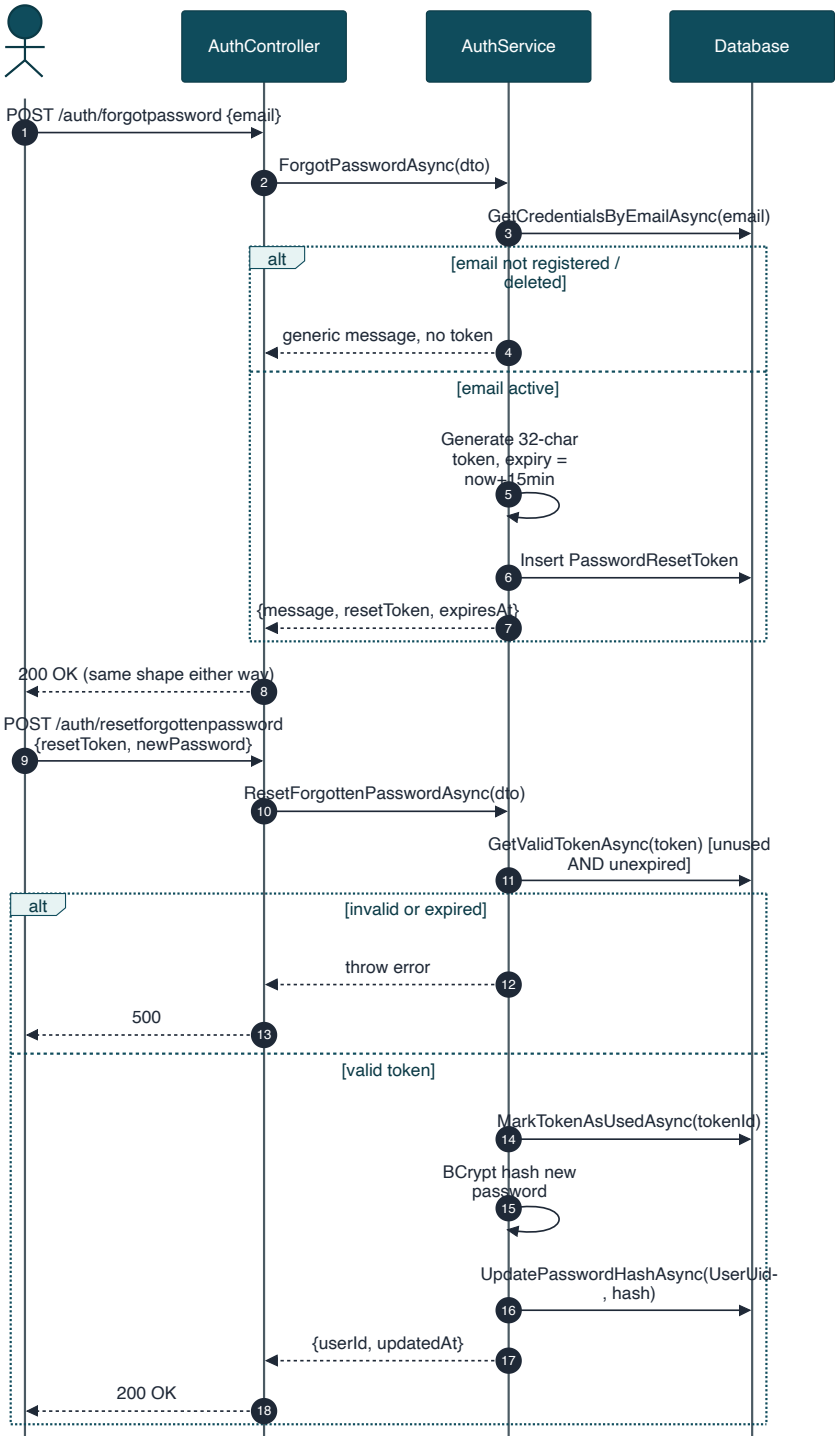

Figure 10. *Sequence — forgot password / reset password (two-step flow).*

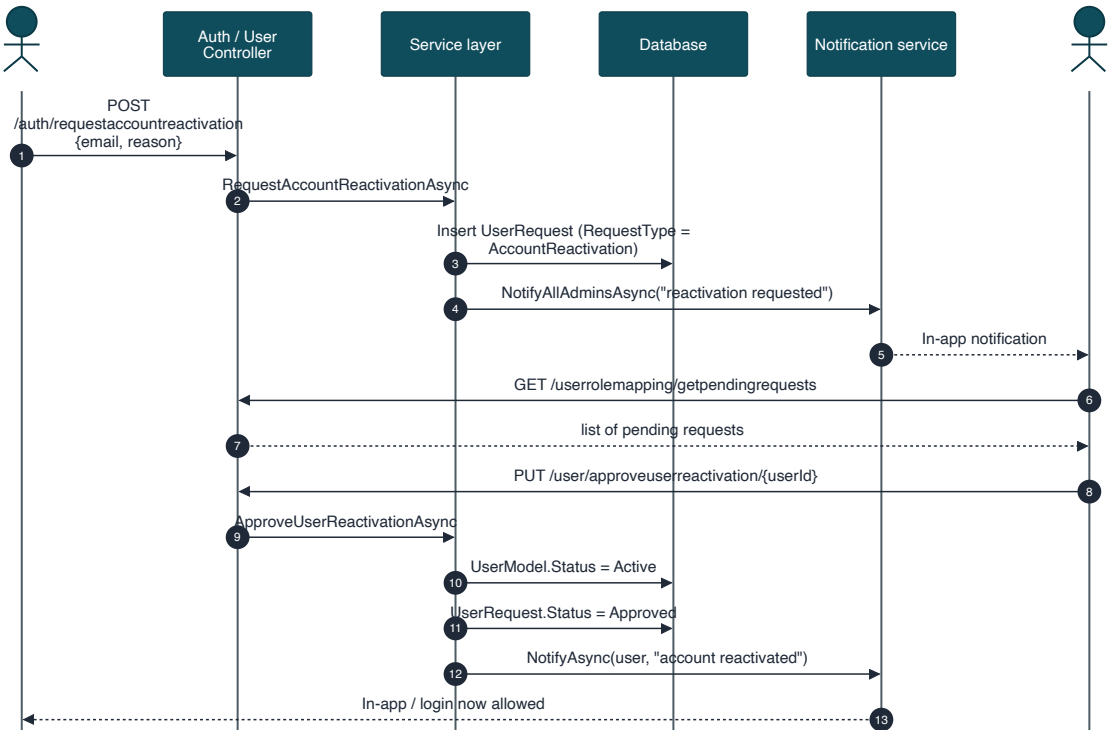

Figure 11. *Sequence — account reactivation request and approval.*

Endpoint Requirements

FR-AUTH-01 — Login — POST

Endpoint	`POST /api/vaxtrack/v1/auth/login`
Actor	Anonymous
Auth Required	None

Behaviour: Verifies email + password against stored credentials, confirms the account is not soft-deleted or Disabled, and issues a signed JWT (claims: user id, email, name, role, unique token id) with a configured expiry.

Business Rules: BR-19 (no enumeration), BR-20 (disabled accounts blocked post-verification), BR-24 (BCrypt only).

Error Cases: Wrong password or unknown email → generic failure. Disabled account → 403 `ACCOUNT_DISABLED` with reason (only after password is verified).

FR-AUTH-02 — Logout POST

Endpoint	`POST /api/vaxtrack/v1/auth/logout`
Actor	Any authenticated user
Auth Required	Bearer JWT

Behaviour: Extracts the token's unique id and expiry from the already-validated request and records it as revoked. Every subsequent authenticated request is checked against this revocation list before it reaches a controller, so the token becomes unusable immediately — even though JWTs are otherwise stateless.

Business Rules: BR-25 (server-side revocation on logout).

FR-AUTH-03 — Forgot Password POST

Endpoint	`POST /api/vaxtrack/v1/auth/forgotpassword`
Actor	Anonymous
Auth Required	None

Behaviour: Step 1 of the recovery flow. Issues a single-use reset token with a 15-minute expiry when the email belongs to an active account. Returns the same generic acknowledgement whether or not the email is registered.

Business Rules: BR-19, BR-21 (15-minute single-use token).

FR-AUTH-04 — Reset Forgotten Password POST

Endpoint	`POST /api/vaxtrack/v1/auth/resetforgottenpassword`
Actor	Anonymous
Auth Required	None

Behaviour: Step 2 of the recovery flow. Validates the reset token (must exist, be unused, and be unexpired), marks it used, then hashes and stores the new password.

Business Rules: BR-21, BR-24.

Error Cases: Invalid, expired, or already-used token → rejected.

FR-AUTH-05 — Request Account Reactivation **POST**

Endpoint	`POST /api/vaxtrack/v1/auth/requestaccountreactivation`
Actor	Anonymous (a disabled account cannot log in to reach any other path)
Auth Required	None

Behaviour: The only route back for a disabled account. Records a pending reactivation request (with an optional reason) for a platform admin to review; see FR-USER-13/14 for the approval path.

Business Rules: BR-20, BR-26 (every disable/reactivation decision requires a platform admin).

5.2 Module: User Management

User

Registration, self-service profile management, and the full user lifecycle

This module owns the user's identity and profile data, self-service account management (email, password, profile picture, self-delete), the platform admin's full user-management toolkit (paged directory, disable/reactivate, promote/demote, hard admin delete), and the user account status lifecycle. Every mutating endpoint enforces an ownership check in the service layer: a caller must either own the target account or hold the platform admin role.

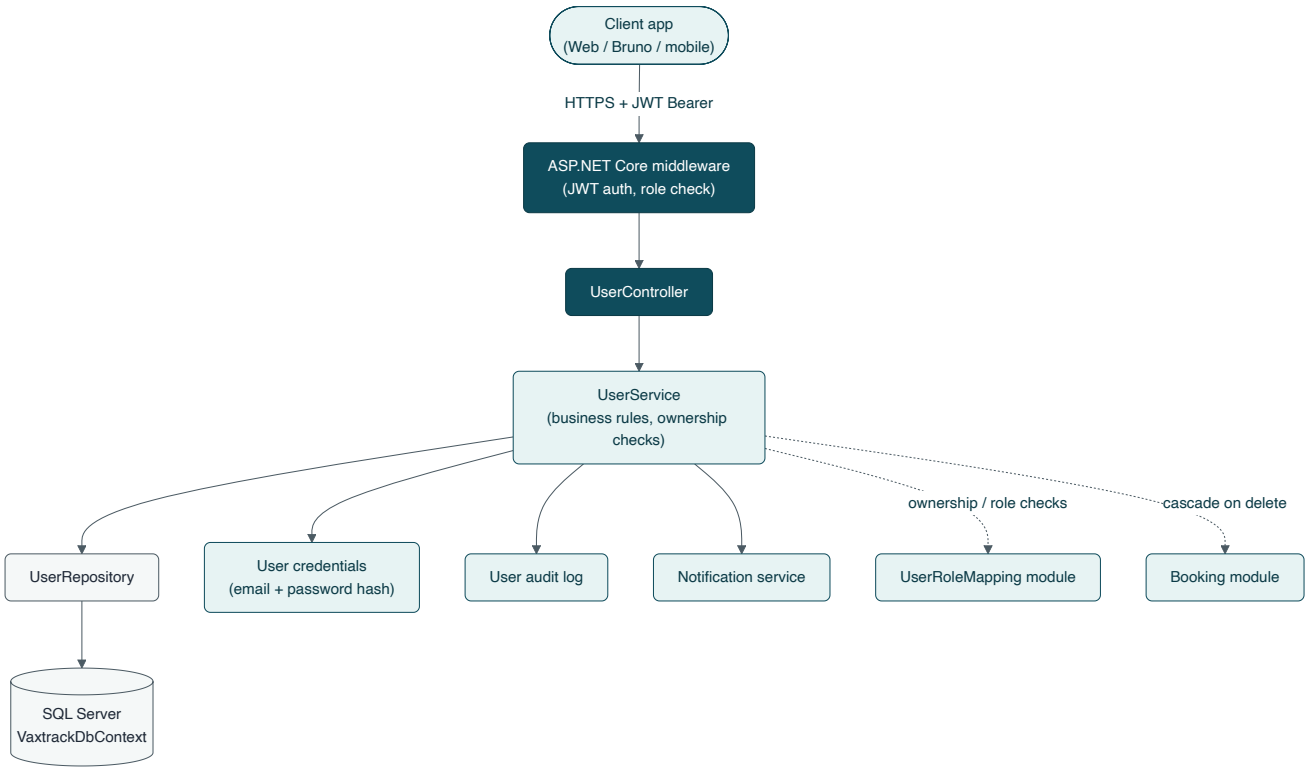

Figure 12. *High-Level Design — User module.*

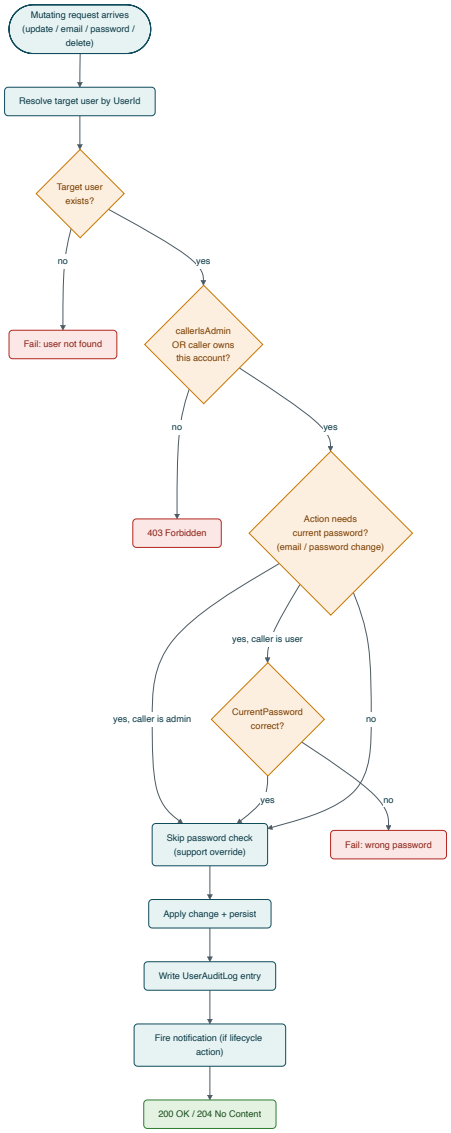

Figure 13. *Low-Level Design — ownership and authorization decision flow for mutating requests.*

Key Flows

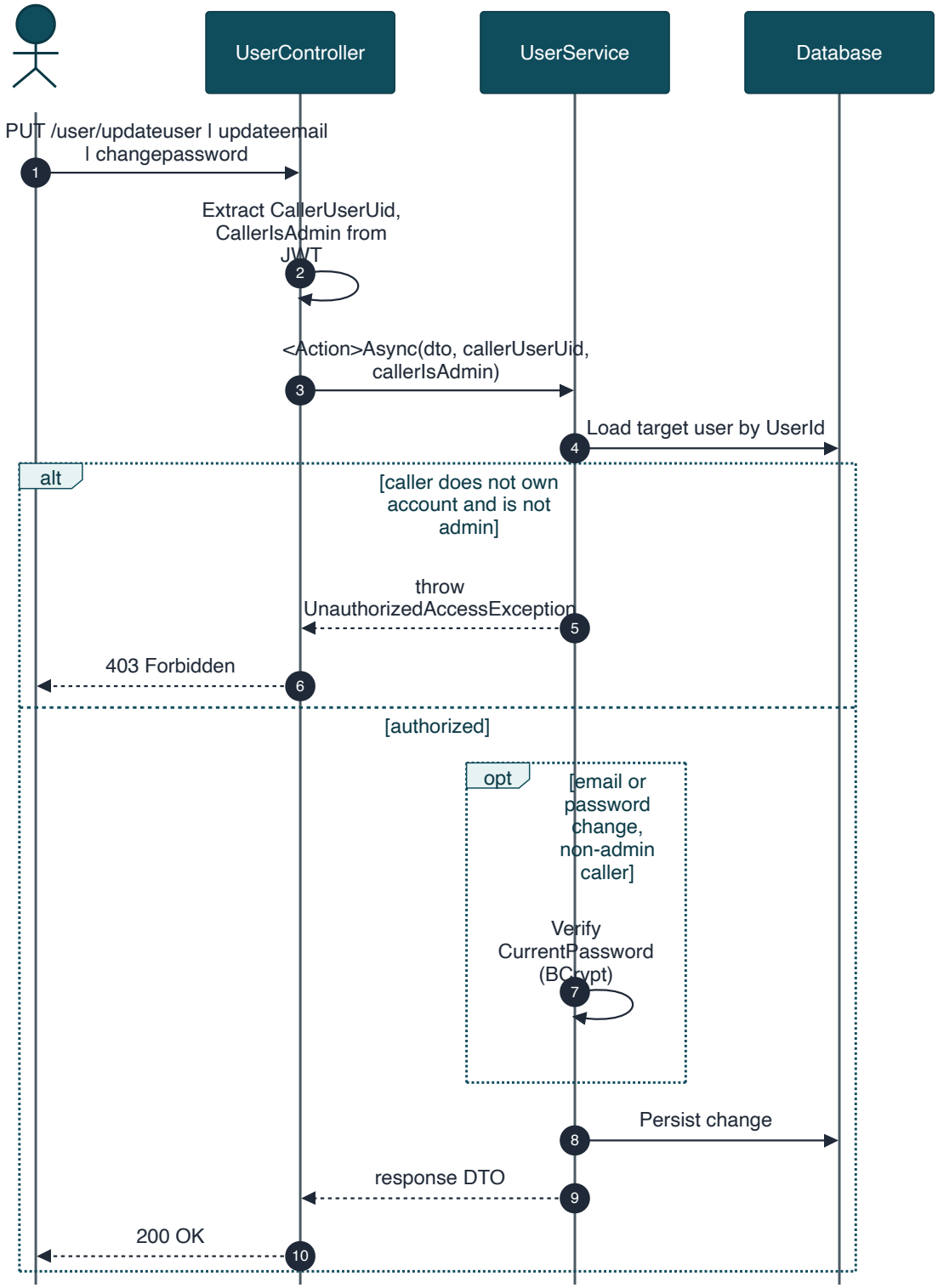

Figure 14. *Sequence — self-service profile, email, and password updates.*

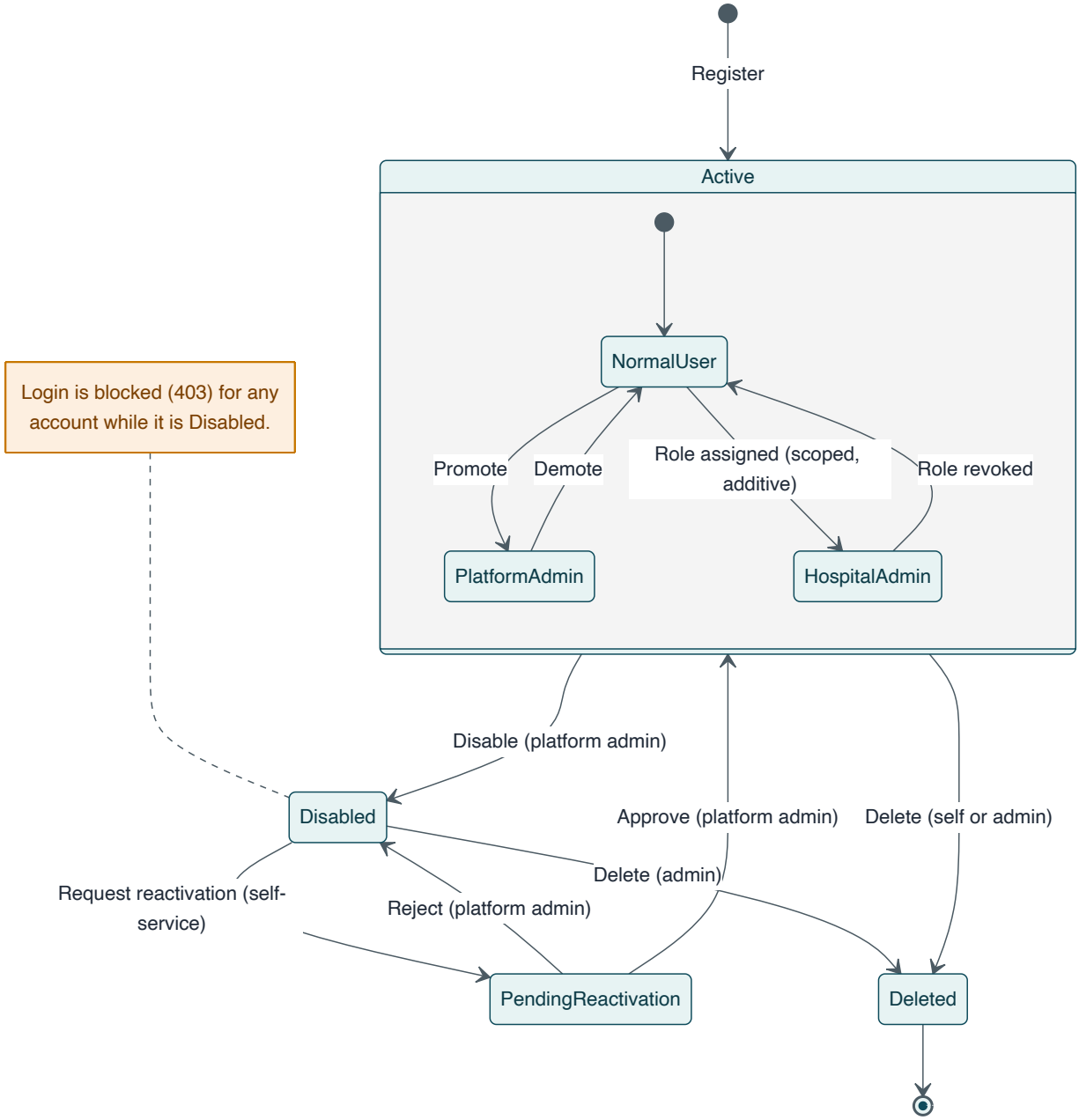

Figure 15. *State diagram — user account lifecycle and platform-role transitions.*

Endpoint Requirements

FR-USER-01 — Create User (Registration) POST

Endpoint	`POST /api/vaxtrack/v1/user/createuser`
Actor	Anonymous

Behaviour: Registers a new normal user. Validates the birthdate is in the past, rejects an email already in active use, computes age at registration, and stores a BCrypt password hash.

Business Rules: BR-16, BR-14, BR-24.

FR-USER-02 — Update User PUT

Endpoint	`PUT /api/vaxtrack/v1/user/updateuser`
Actor	Owner or platform admin

Behaviour: Updates mutable profile fields (name, gender, phone, address, pin code, profile picture path). Identity fields (UserId, UserUid, birthdate, age, role, email) are immutable through this endpoint.

Error Cases: Caller does not own the account and is not admin → 403.

FR-USER-03 — Update Email PUT

Endpoint	`PUT /api/vaxtrack/v1/user/updateemail`
Actor	Owner or platform admin

Behaviour: Non-admin callers must supply their current password; admins may override without it (support/recovery use case). New email must not already belong to another active account.

Business Rules: BR-14, BR-15.

FR-USER-04 — Change Password PUT

Endpoint	`PUT /api/vaxtrack/v1/user/changepassword`
Actor	Owner or platform admin

Behaviour: Same non-admin-password-required pattern as Update Email. Distinct from the anonymous forgot-password flow — this requires an active session.

Business Rules: BR-15, BR-24.

FR-USER-05 — Upload Profile Picture — POST

Endpoint	`POST /api/vaxtrack/v1/user/uploadprofilepicture`
Actor	Owner or platform admin

Behaviour: Accepts a multipart file upload and stores the resulting path against the user's profile.

FR-USER-06 — Remove Profile Picture — PUT

Endpoint	`PUT /api/vaxtrack/v1/user/removeprofilepicture/{userId}`
Actor	Owner or platform admin

Behaviour: Clears the stored profile picture path.

FR-USER-07 — Get All Users — GET

Endpoint	`GET /api/vaxtrack/v1/user/getallusers`
Actor	Platform admin only

Behaviour: Returns the full user directory. Superseded operationally by the paged variant (FR-USER-08) for the Users Management screen.

FR-USER-08 — Get Users (Paged & Filtered) — GET

Endpoint	`GET /api/vaxtrack/v1/user/getuserspaged`
Actor	Platform admin only

Behaviour: Filterable by name, phone, user id/uid, role, and vaccination status; sortable; paginated. Backs the platform admin's Users Management screen.

FR-USER-09 — Get User Profile Data — GET

Endpoint	`GET /api/vaxtrack/v1/user/getuserprofiledata/{userId}`
Actor	Owner or platform admin

Behaviour: Returns the full profile, including hospital-admin flag, vaccination status, and account status. A non-owner, non-admin caller is rejected — this prevents PII harvesting.

FR-USER-10 — Delete My Account — DELETE

Endpoint	`DELETE /api/vaxtrack/v1/user/deletemyaccount`
Actor	Any authenticated user (self only)

Behaviour: Requires the caller's current password. Identity comes from the JWT, not a URL parameter. Cascades: soft-deletes the user, soft-deletes their active bookings (releasing any reserved slots), revokes all role mappings, and soft-deletes their credentials (freeing the email for reuse).

Business Rules: BR-12, BR-14.

FR-USER-11 — Delete User (Admin) — DELETE

Endpoint	`DELETE /api/vaxtrack/v1/user/deleteuser/{userId}`
Actor	Platform admin only

Behaviour: Same cascade as Delete My Account, targeting any user by id.

Business Rules: BR-12.

FR-USER-12 — Disable User — PUT

Endpoint	`PUT /api/vaxtrack/v1/user/disableuser/{userId}`
Actor	Platform admin only

Behaviour: Sets the account to Disabled with a mandatory reason. Blocks all future login attempts (Section 5.1, FR-AUTH-01) until reactivated.

Business Rules: BR-20, BR-26.

FR-USER-13 — Approve User Reactivation — PUT

Endpoint	`PUT /api/vaxtrack/v1/user/approveuserreactivation/{userId}`
Actor	Platform admin only

Behaviour: Resolves a pending reactivation request (see FR-AUTH-05) by restoring the account to Active.

FR-USER-14 — Reject User Reactivation PUT

Endpoint	`PUT /api/vaxtrack/v1/user/rejectuserreactivation/{userId}`
Actor	Platform admin only

Behaviour: Resolves a pending reactivation request by keeping the account Disabled, with an optional comment.

FR-USER-15 — Promote to Admin PUT

Endpoint	`PUT /api/vaxtrack/v1/user/promotetoadmin/{userId}`
Actor	Platform admin only

Behaviour: Grants the platform-wide admin flag to the target user.

FR-USER-16 — Demote from Admin PUT

Endpoint	`PUT /api/vaxtrack/v1/user/demotefromadmin/{userId}`
Actor	Platform admin only

Behaviour: Revokes the platform-wide admin flag from the target user, returning them to normal-user privileges (any scoped hospital-admin roles they separately hold are unaffected).

5.3 Module: Hospital Management

Hospital

Hospital directory, slot capacity, and the disable / reactivate / unregister lifecycle

This module owns each hospital's profile, its slot capacity, and its operational status. Contact-info and capacity changes are available to a platform admin or to that hospital's own scoped hospital-admin — a hospital-admin for one hospital has no visibility into changing another's data. Permanently removing a hospital is deliberately a two-party process: a platform admin initiates it, and the hospital's own admin must separately re-authenticate to authorize it (or decline it) — see BR-11 and Figure 20.

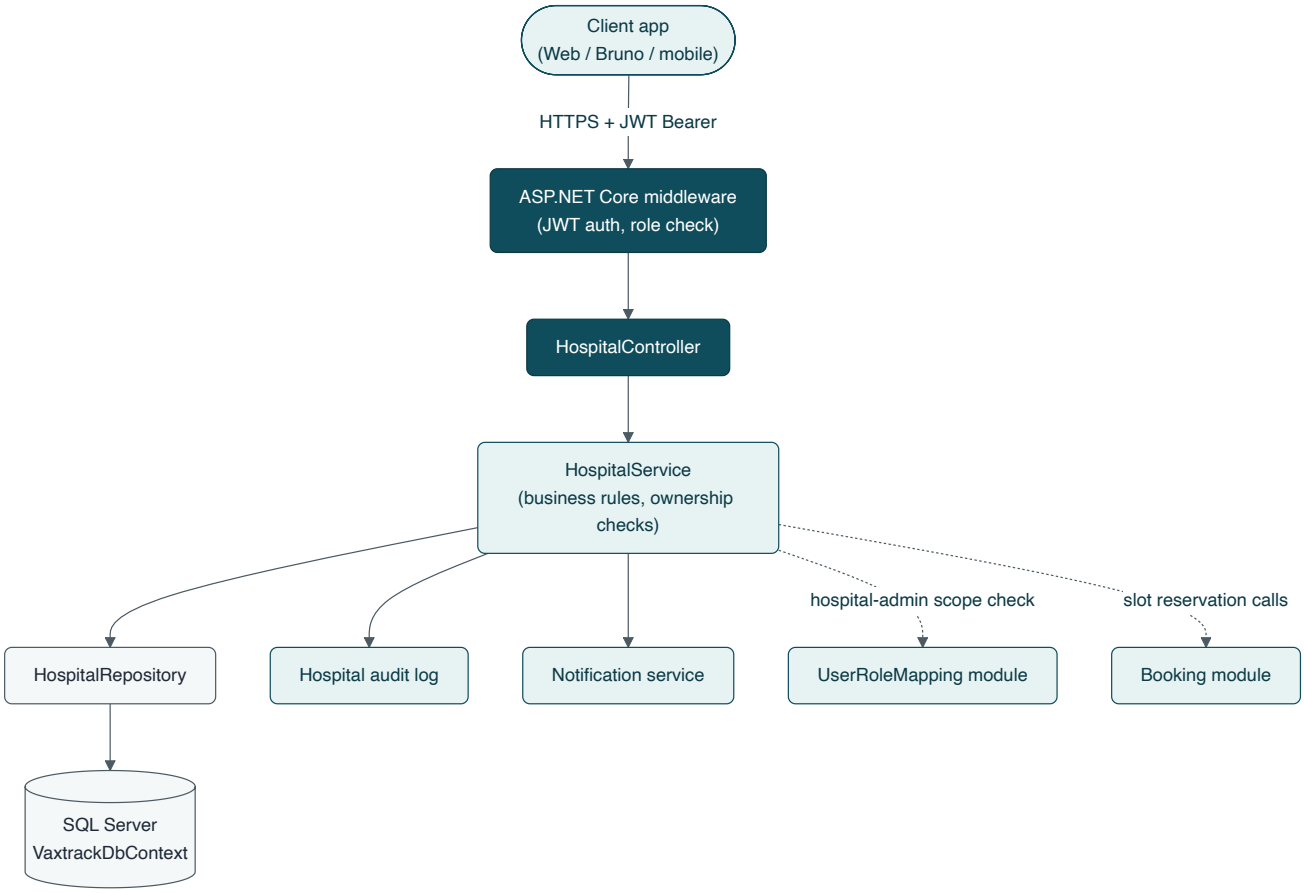

Figure 16. *High-Level Design — Hospital module.*

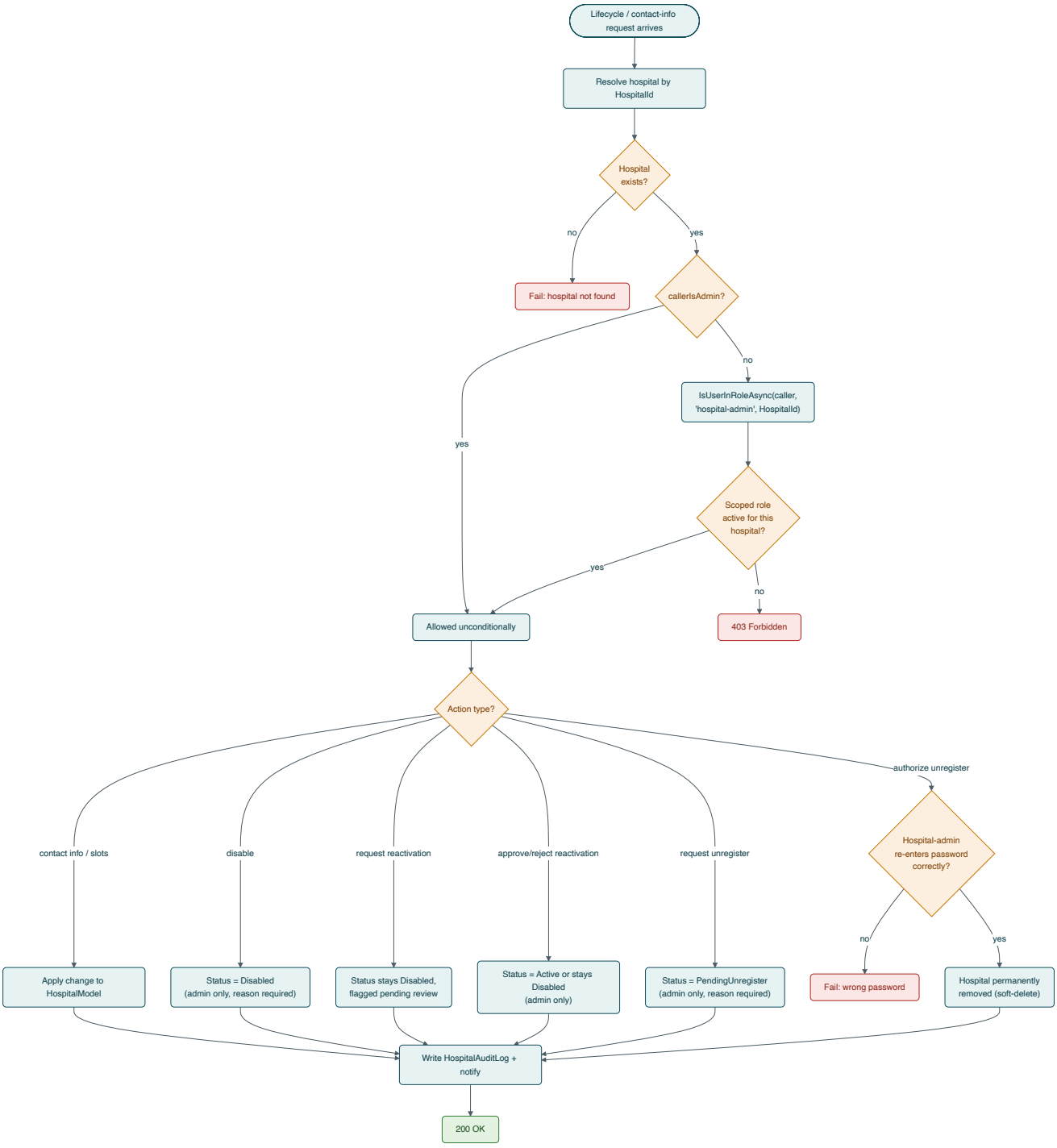

Figure 17. *Low-Level Design — hospital-admin scope check and lifecycle action routing.*

Key Flows

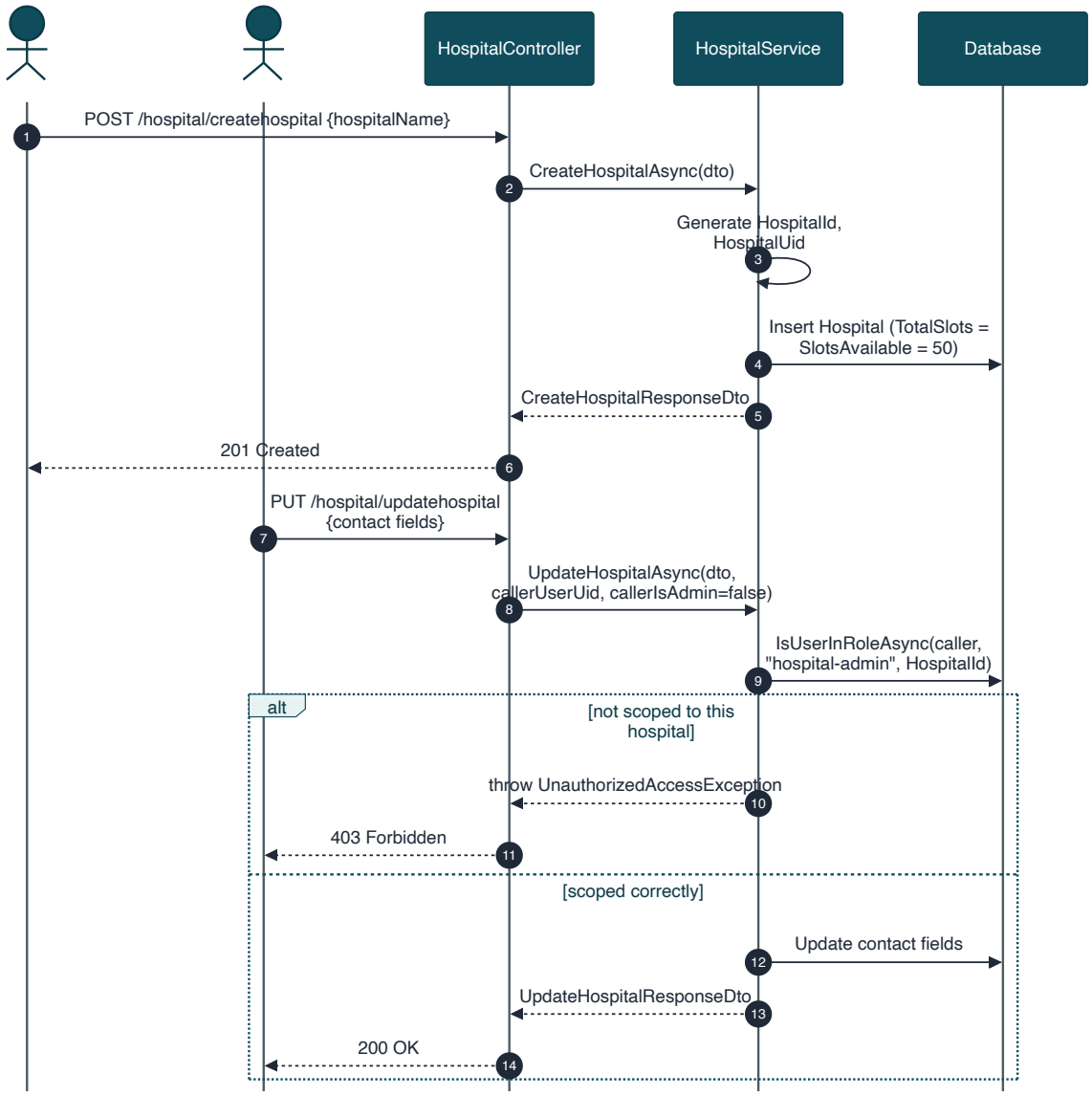

Figure 18. *Sequence — create hospital and scoped contact-info update.*

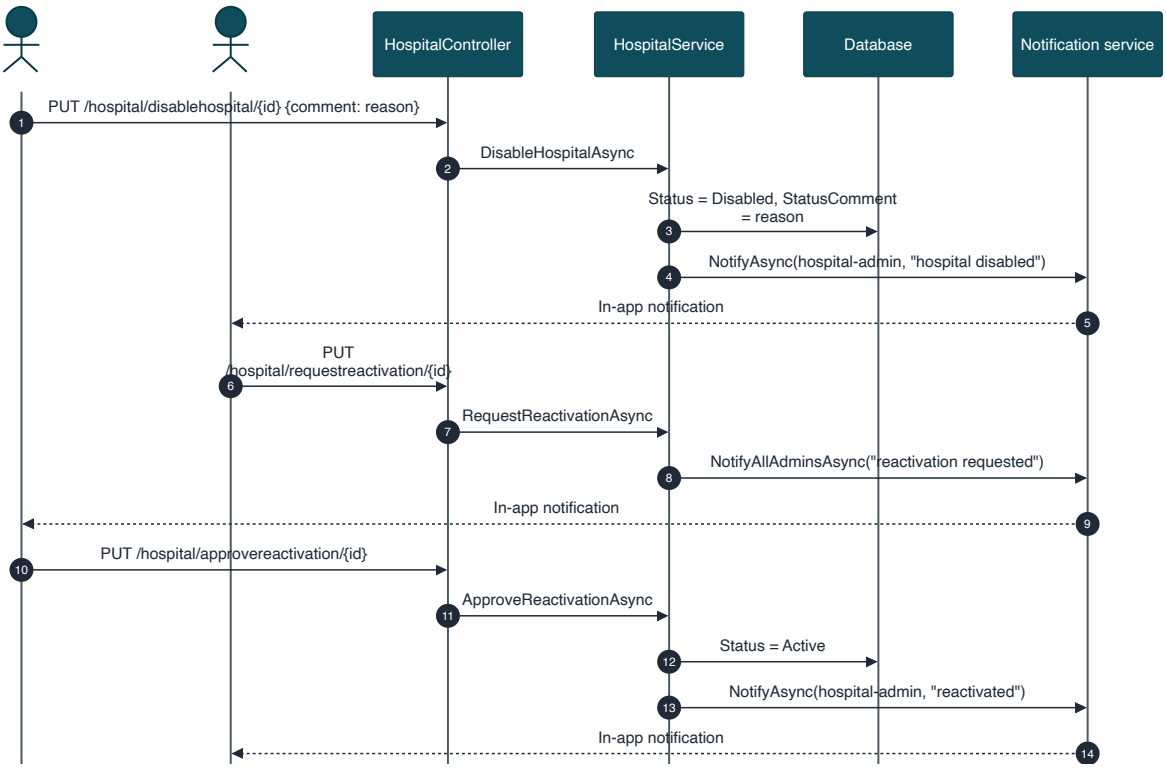

Figure 19. *Sequence — disable and reactivation-request lifecycle.*

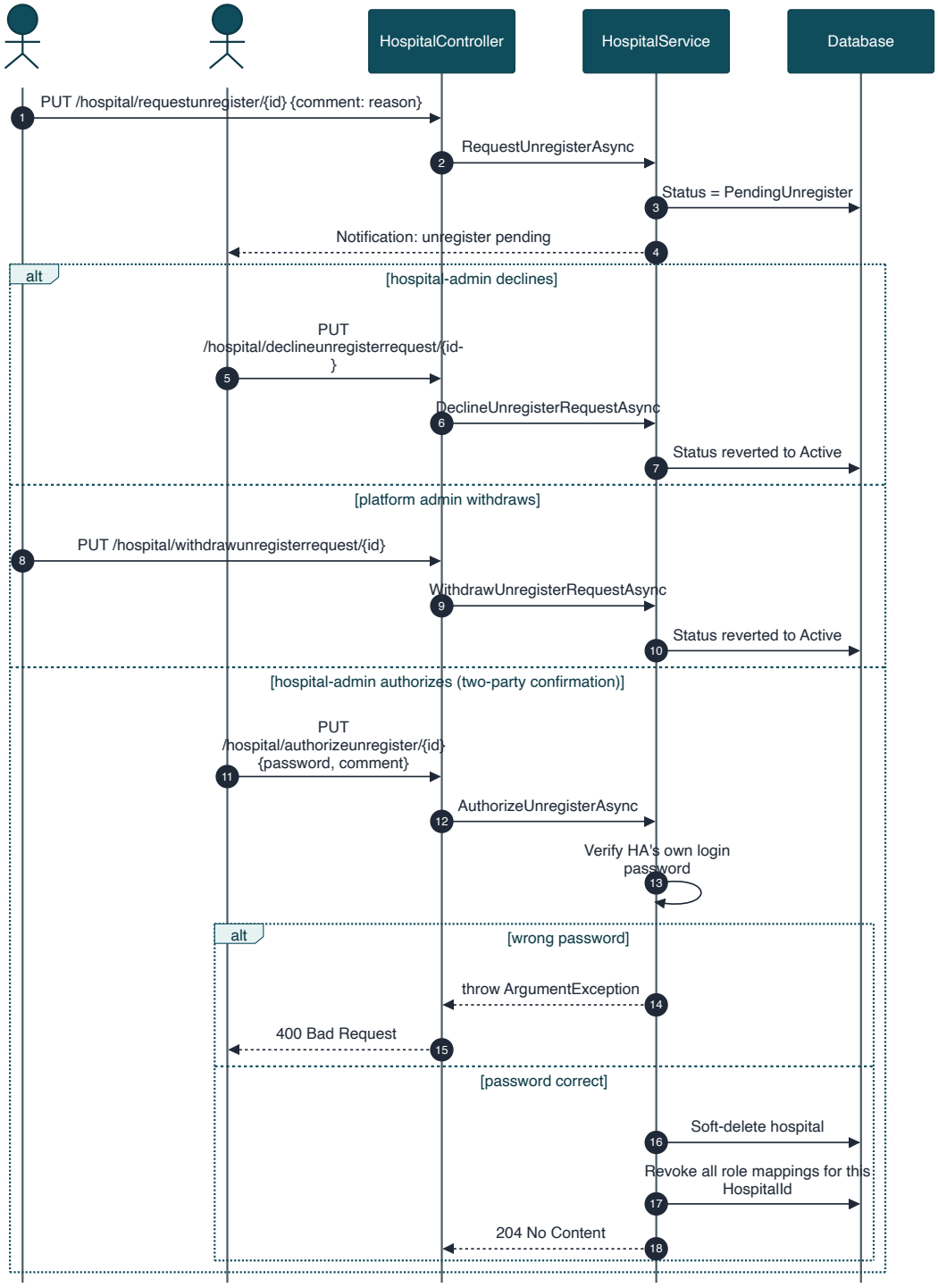

Figure 20. *Sequence — two-party unregister confirmation flow.*

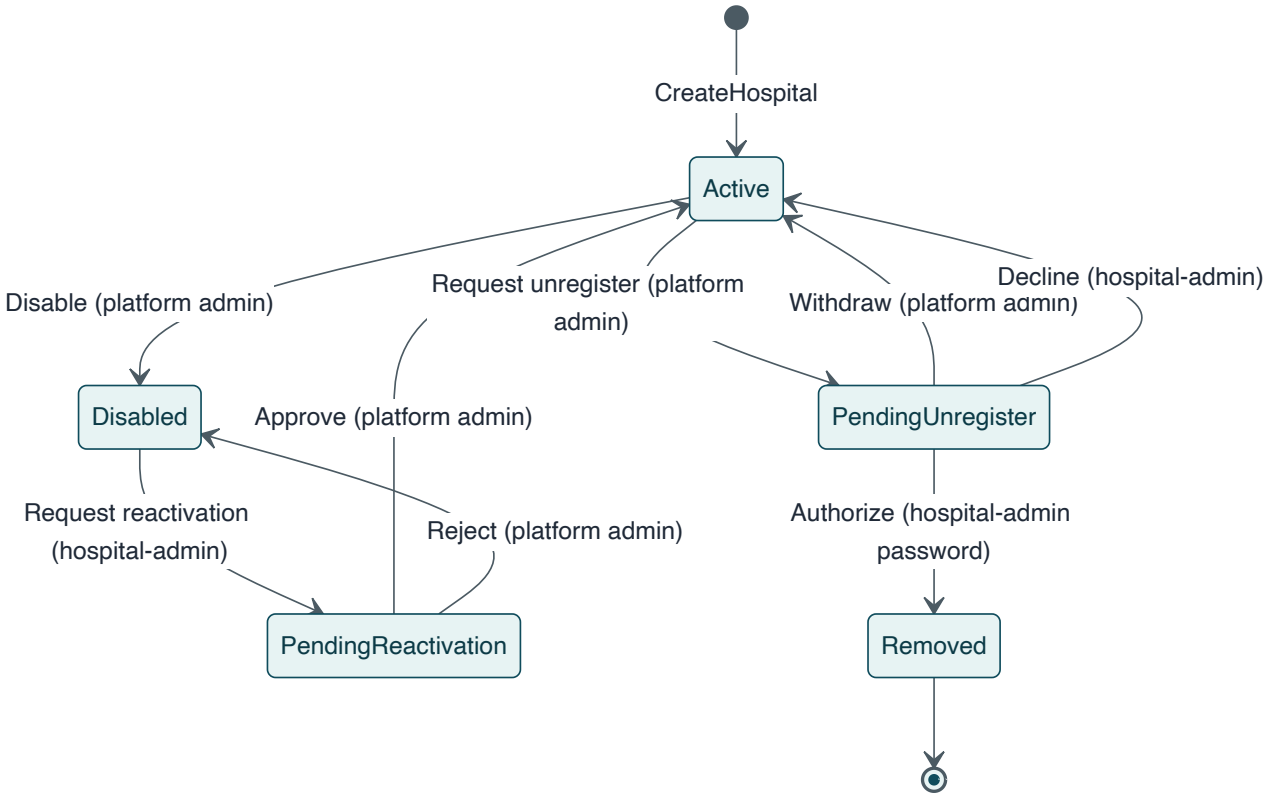

Figure 21. *State diagram — hospital operational status lifecycle.*

Endpoint Requirements

FR-HOSP-01 — Create Hospital — POST

Endpoint	`POST /api/vaxtrack/v1/hospital/createhospital`
Actor	Platform admin only

Behaviour: Registers a new hospital with a default capacity of 50 total / 50 available slots. Contact fields start empty and are filled in via Update Hospital.

FR-HOSP-02 — Update Hospital — PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/updatehospital`
Actor	Platform admin or that hospital's scoped hospital-admin

Behaviour: Updates address, pin code, phone, and email. Name, id, and slot fields are immutable through this endpoint.

Business Rules: BR-07, BR-08.

FR-HOSP-03 — Update Total Slots PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/updatetotalslots/{hospitalId}/{totalSlots}`
Actor	Platform admin or scoped hospital-admin

Behaviour: Sets the hospital's maximum capacity. If the new total falls below the current available count, available is clamped down to match — available can never exceed total.

Business Rules: BR-08.

FR-HOSP-04 — Update Available Slots PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/updateavailableslots/{hospitalId}/{slotsToUpdate}`
Actor	Platform admin, scoped hospital-admin, or the Booking module internally

Behaviour: Applies a signed delta to the available count (e.g. -1 to reserve, +1 to release). Rejects any change that would push available below zero or above total. Called internally by the Booking module on every reserve/release — those internal calls bypass the human authorization check.

Business Rules: BR-08.

FR-HOSP-05 — Get Hospital By Id GET

Endpoint	`GET /api/vaxtrack/v1/hospital/gethospitalbyid/{hospitalId}`
Actor	Any authenticated user

FR-HOSP-06 — Get All Hospitals GET

Endpoint	`GET /api/vaxtrack/v1/hospital/getallhospitals`
Actor	Any authenticated user

FR-HOSP-07 — Disable Hospital PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/disablehospital/{hospitalId}`
Actor	Platform admin only

Behaviour: Sets status to Disabled with a mandatory reason.

Business Rules: BR-26.

FR-HOSP-08 — Request Reactivation PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/requestreactivation/{hospitalId}`
Actor	That hospital's own scoped hospital-admin

FR-HOSP-09 — Approve / Reject Reactivation PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/approvereactivation/{hospitalId}` | `PUT /.../rejectreactivation/{hospitalId}`
Actor	Platform admin only

Behaviour: Resolves a pending reactivation request — approve restores Active status, reject keeps the hospital Disabled.

FR-HOSP-10 — Request Unregister PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/requestunregister/{hospitalId}`
Actor	Platform admin only

Behaviour: Moves the hospital to PendingUnregister with a mandatory reason. Does not remove the hospital by itself — see FR-HOSP-13.

Business Rules: BR-11.

FR-HOSP-11 — Withdraw Unregister Request PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/withdrawunregisterrequest/{hospitalId}`
Actor	Platform admin only

Behaviour: The initiating platform admin reverses their own pending unregister request, restoring Active status.

FR-HOSP-12 — Decline Unregister Request PUT

Endpoint	`PUT /api/vaxtrack/v1/hospital/declineunregisterrequest/{hospitalId}`
Actor	That hospital's own scoped hospital-admin

Behaviour: The hospital's own admin pushes back on a pending unregister, restoring Active status.

FR-HOSP-13 — Authorize Unregister **PUT**

Endpoint	`PUT /api/vaxtrack/v1/hospital/authorizeunregister/{hospitalId}`
Actor	That hospital's own scoped hospital-admin (or the platform admin if no hospital-admin is assigned)

Behaviour: The second-party confirmation step. The caller re-enters their own login password; on success the hospital is permanently removed (soft-deleted) and every role mapping scoped to it is revoked.

Business Rules: BR-11.

Error Cases: Wrong password → 400 Bad Request.

FR-HOSP-14 — Get Hospital Audit Trail **GET**

Endpoint	`GET /api/vaxtrack/v1/hospital/gethospitalauditttrail/{hospitalId}`
Actor	Platform admin or that hospital's scoped hospital-admin

Behaviour: Returns the full, permanent history of lifecycle actions taken against this hospital — actor, action type, comment, and timestamp for each.

5.4 Module: Booking & Vaccination Lifecycle

Booking

The two-dose booking journey, approvals, cancellations, audit trail, and certificates

This is the platform's core module: one booking record tracks a single beneficiary's Dose 1 and Dose 2 journey from request through completion, cancellation, or rejection. Every dose-level action is ownership- or role-checked, slot capacity is reserved and released automatically via the Hospital module, and every action is written to a permanent audit trail. A completed vaccination produces a public, unauthenticated certificate that mirrors how a real vaccination QR code works.

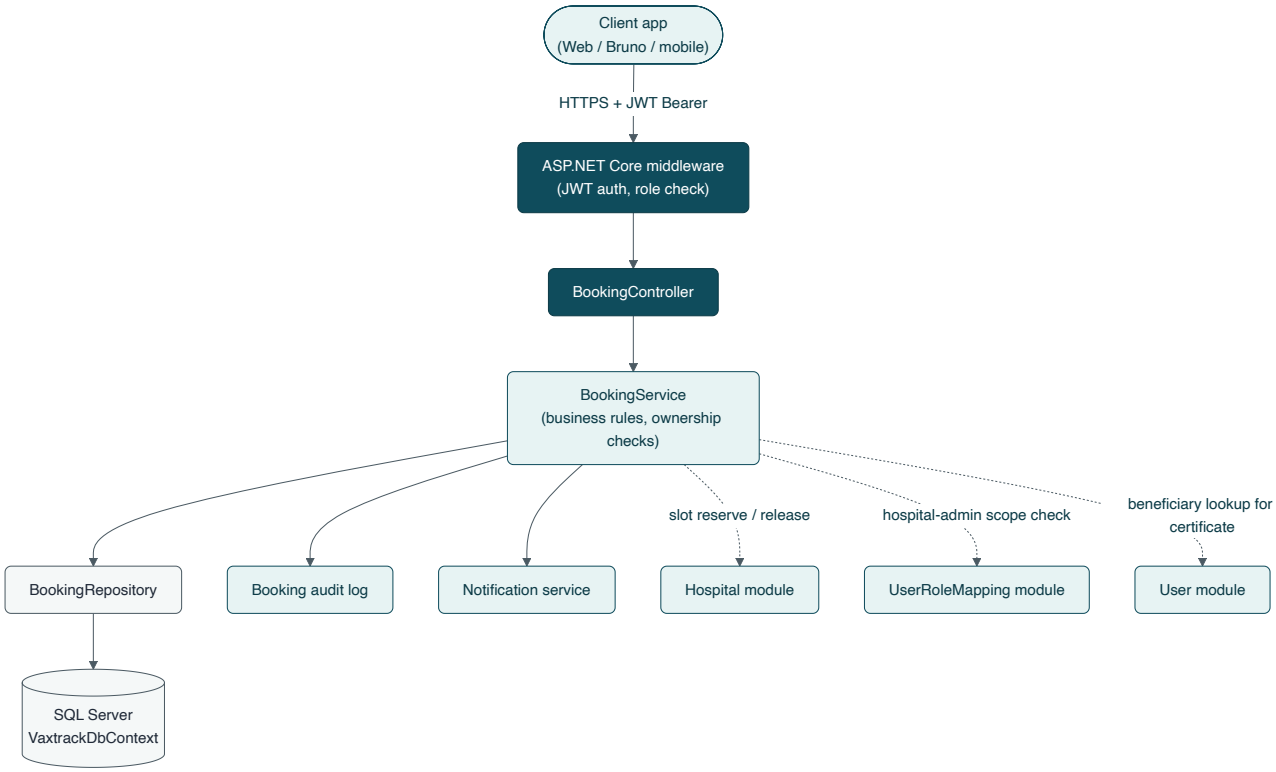

Figure 22. *High-Level Design — Booking module.*

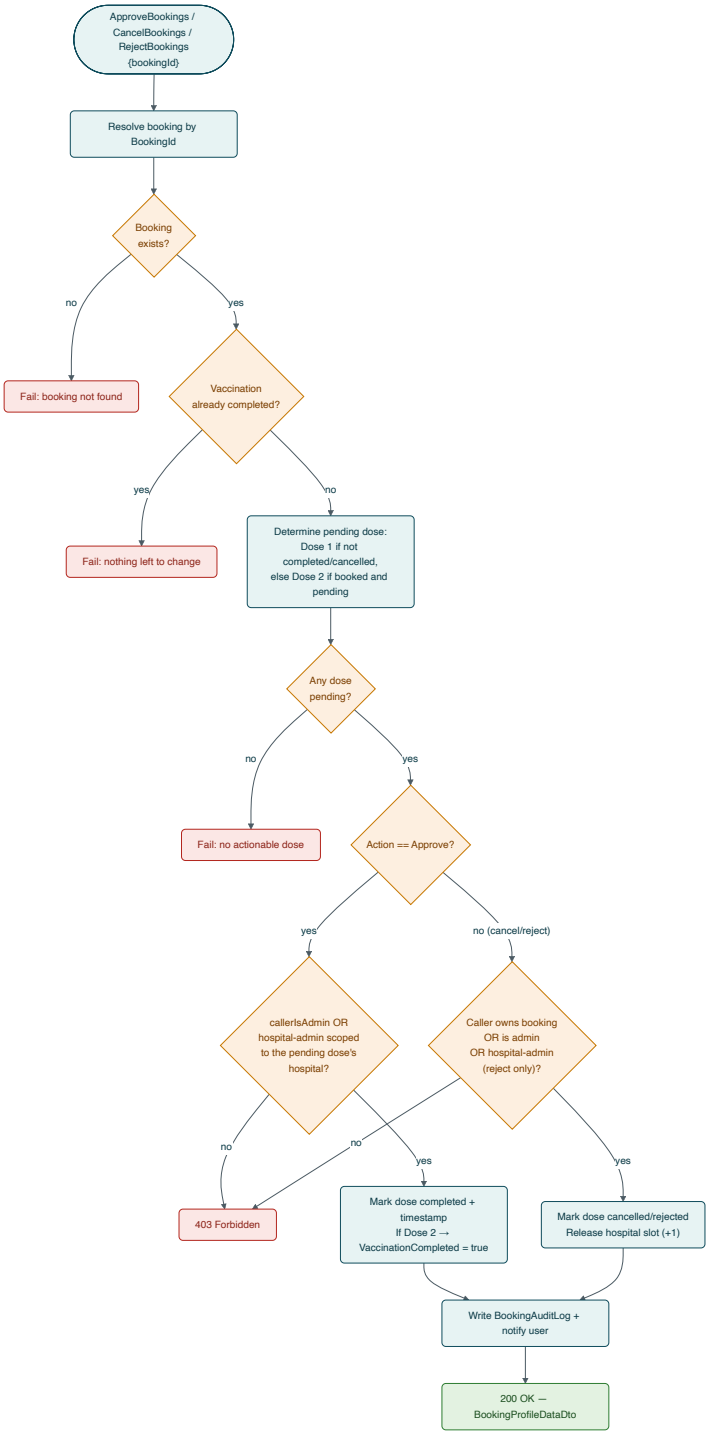

Figure 23. *Low-Level Design — approve/cancel/reject decision flow.*

Key Flows

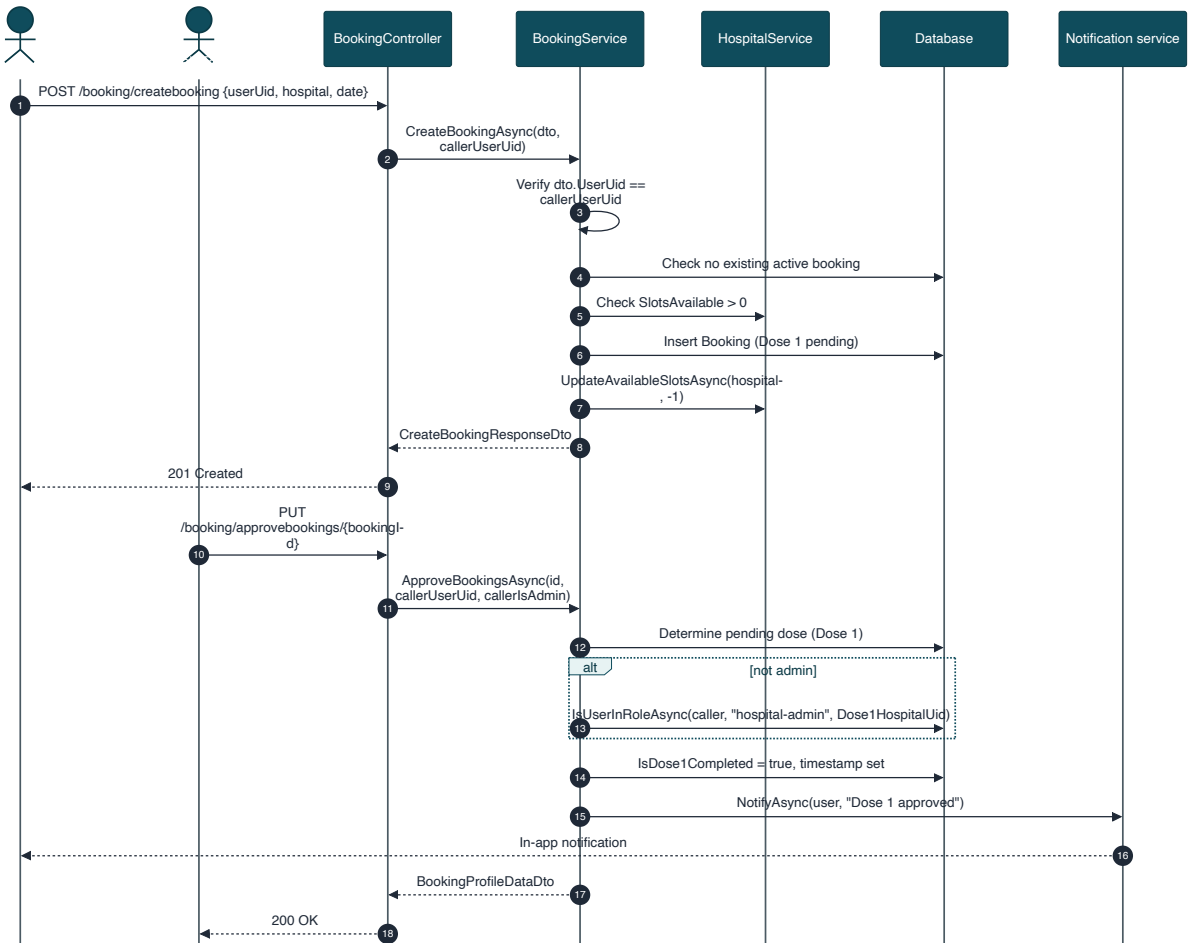

Figure 24. *Sequence — create Dose 1 booking and approval.*

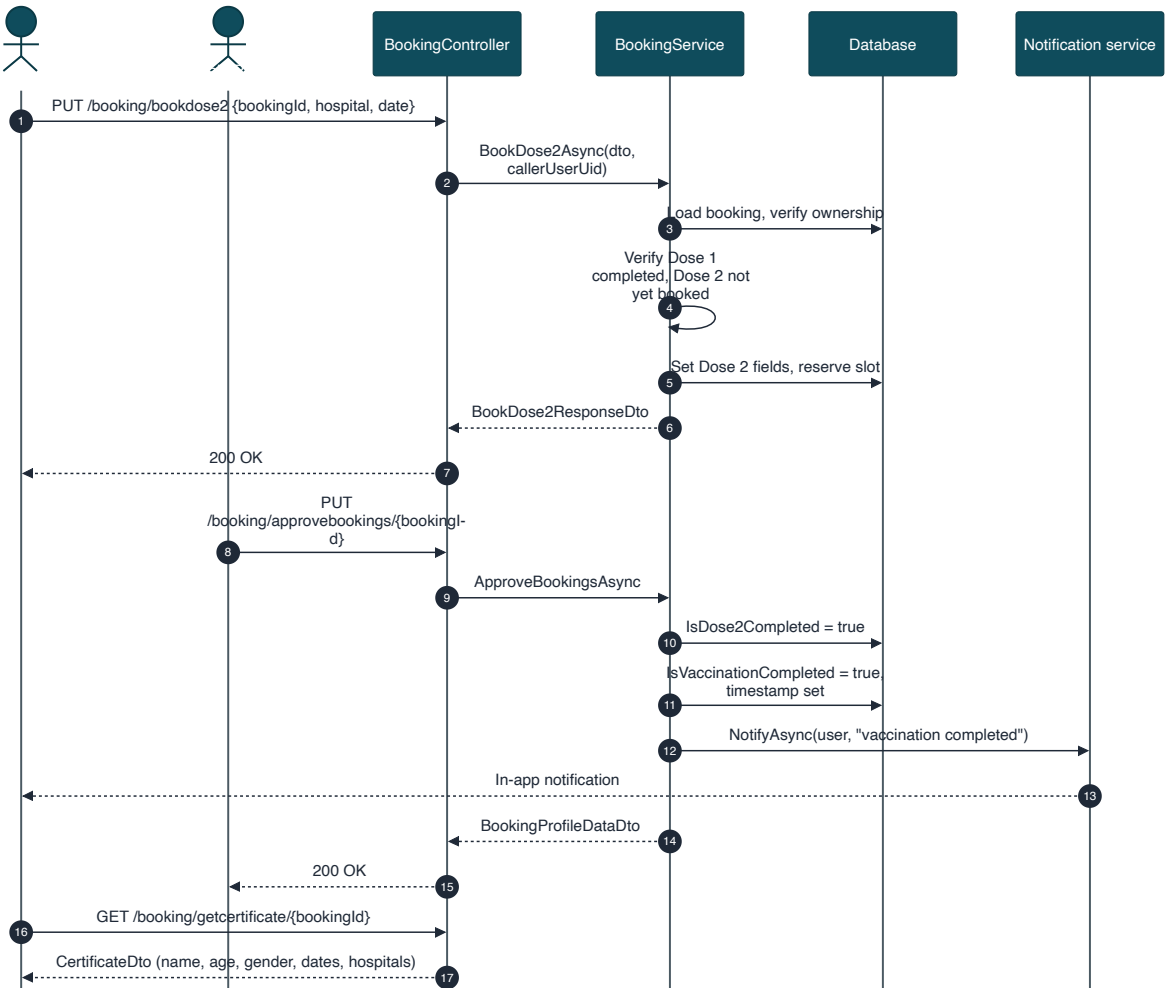

Figure 25. *Sequence — Dose 2 booking through vaccination completion and certificate.*

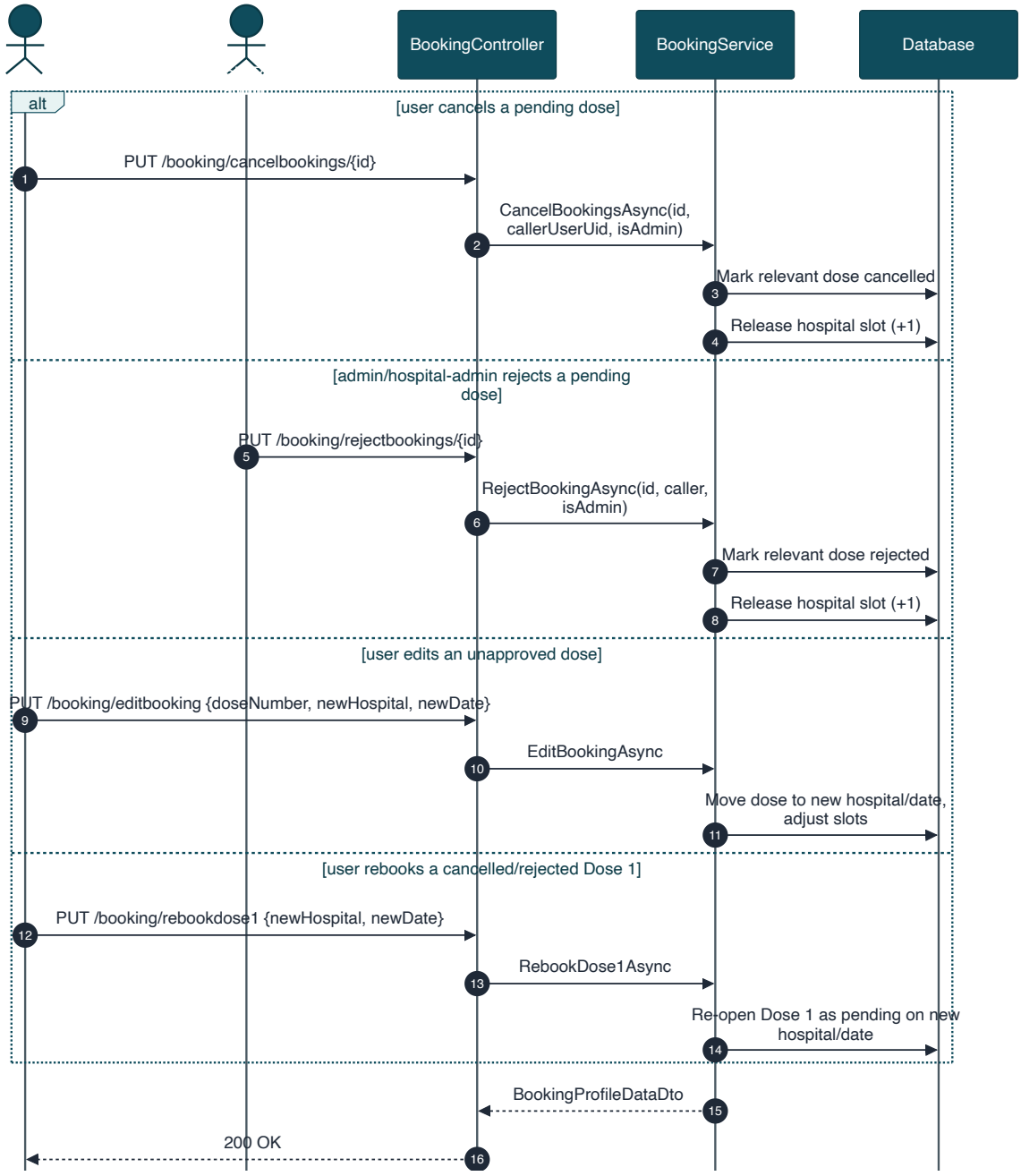

Figure 26. *Sequence — cancel, reject, edit, and rebook variants.*

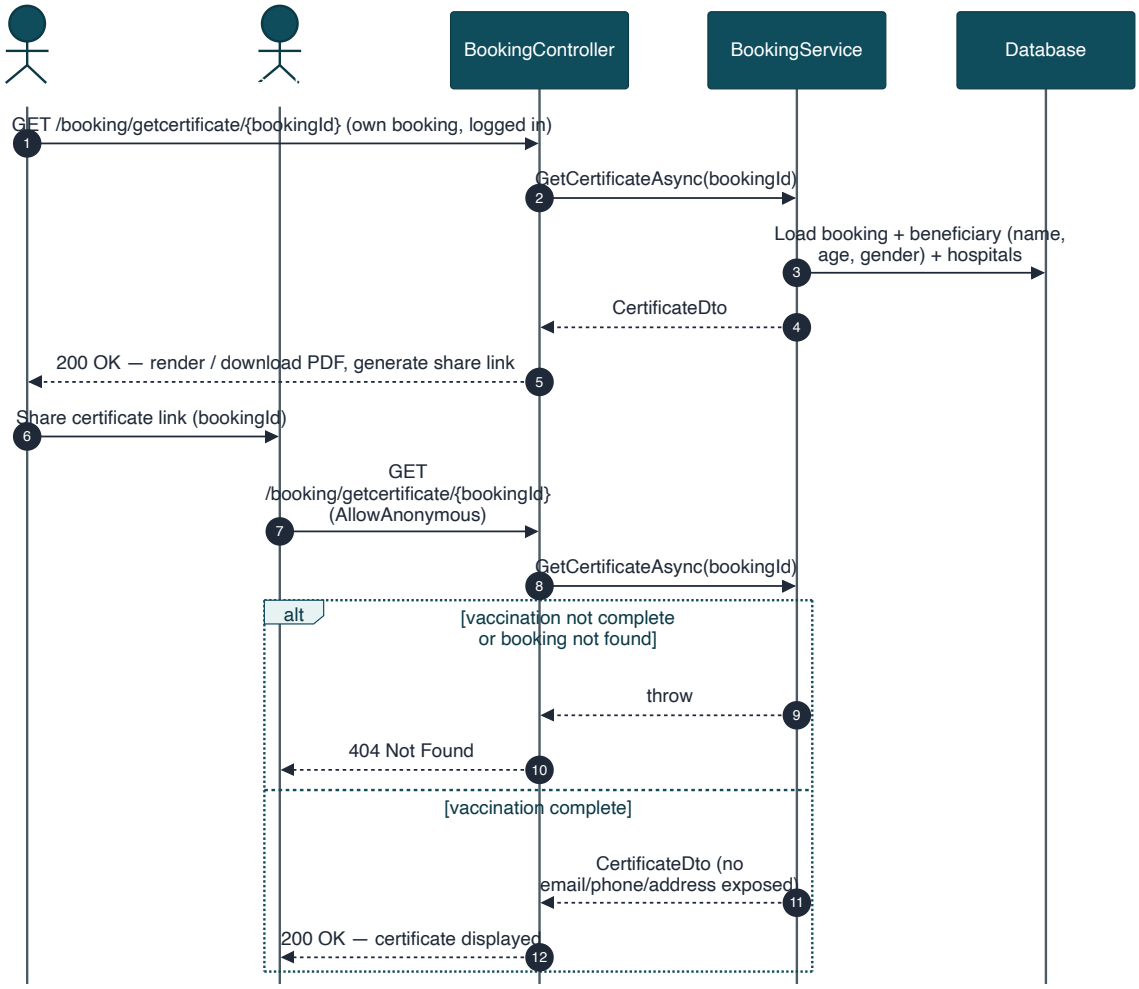

Figure 27. *Sequence — certificate download and public verification.*

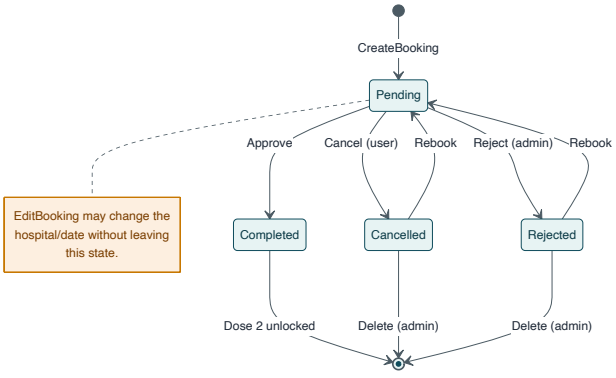

Figure 28. *State diagram — Dose 1 lifecycle.*

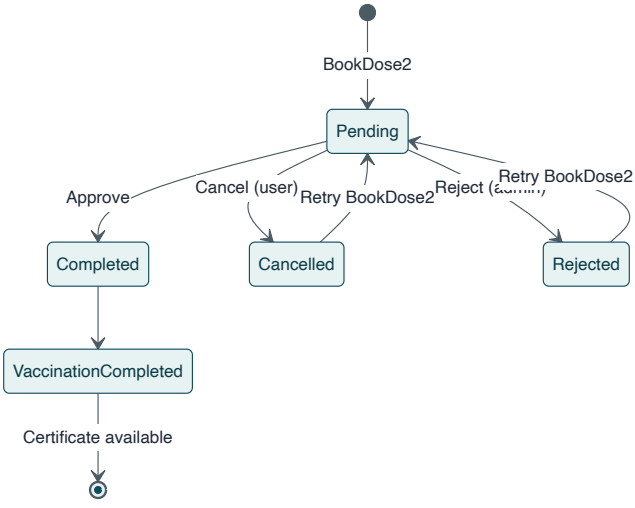

Figure 29. *State diagram — Dose 2 and vaccination completion.*

Endpoint Requirements

FR-BOOK-01 — Create Booking (Book Dose 1) POST

Endpoint	`POST /api/vaxtrack/v1/booking/createbooking`
Actor	Any authenticated user (self only)

Behaviour: Verifies the target user matches the caller, the requested date is in the future, the caller has no existing active booking, and the hospital has capacity — then creates the booking and reserves one slot.

Business Rules: BR-01, BR-02, BR-13.

Error Cases: Impersonation attempt → 403. Past date, duplicate booking, or no capacity → rejected.

FR-BOOK-02 — Update Booking (Admin Override) PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/updatebooking`
Actor	Platform admin only

Behaviour: Full-field override, including dose-completion flags and timestamps — used for manual data correction. Deliberately restricted to platform admin since it can mark a dose "completed" without the normal approval path.

Business Rules: BR-09.

FR-BOOK-03 — Book Dose 2 PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/bookdose2`
Actor	Booking owner

Behaviour: Only permitted once Dose 1 is completed and Dose 2 has not already been booked. Dose 2 may be at a different hospital than Dose 1.

Business Rules: BR-03, BR-04.

FR-BOOK-04 — Edit Booking PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/editbooking`
Actor	Booking owner

Behaviour: Moves an unapproved dose (1 or 2) to a new hospital and/or date without cancelling and recreating the booking.

FR-BOOK-05 — Rebook Dose 1 PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/rebookdose1`
Actor	Booking owner

Behaviour: Re-opens Dose 1 as pending on a new hospital/date after it was cancelled or rejected, rather than requiring a brand-new booking record.

FR-BOOK-06 — Approve Bookings PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/approvebookings/{bookingId}`
Actor	Platform admin, or scoped hospital-admin for the pending dose's hospital

Behaviour: Determines whichever dose is currently pending and marks it completed with a timestamp. Approving Dose 2 also marks the whole booking as vaccination-completed.

Business Rules: BR-03, BR-07.

Error Cases: Already fully vaccinated, or no pending dose → rejected. Wrong-hospital hospital-admin → 403.

FR-BOOK-07 — Cancel Bookings PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/cancelbookings/{bookingId}`
Actor	Booking owner or platform admin

Behaviour: Cancels whichever dose is currently pending and releases its reserved slot. A completed dose can never be cancelled.

Business Rules: BR-05, BR-06.

FR-BOOK-08 — Reject Bookings PUT

Endpoint	`PUT /api/vaxtrack/v1/booking/rejectbookings/{bookingId}`
Actor	Platform admin or scoped hospital-admin

Behaviour: Administratively rejects the pending dose (distinct from a user-initiated cancel, for audit-trail clarity) and releases the slot.

Business Rules: BR-06, BR-07.

FR-BOOK-09 — Get Booking Audit Trail GET

Endpoint	`GET /api/vaxtrack/v1/booking/getbookingauditttrail/{bookingId}`
Actor	Booking owner or platform admin

Behaviour: Returns the permanent history of every action taken on this booking — dose number, action type, actor, role, comment, and timestamp.

FR-BOOK-10 — Get Actionable Bookings GET

Endpoint	`GET /api/vaxtrack/v1/booking/getactionablebookings`
Actor	Any authenticated user

Behaviour: Bookings still needing action, scoped internally to the caller's own hospital-admin assignment(s), or all such bookings for a platform admin. A caller with no assignment simply receives an empty list.

FR-BOOK-11 — Get Booking By Id GET

Endpoint	`GET /api/vaxtrack/v1/booking/getbookingbybookingid/{bookingId}`
Actor	Booking owner or platform admin

FR-BOOK-12 — Get Bookings By User GET

Endpoint	`GET /api/vaxtrack/v1/booking/getbookingsbyuserid/{userId}`
Actor	That user or platform admin

FR-BOOK-13 — Get Bookings By Hospital GET

Endpoint	`GET /api/vaxtrack/v1/booking/getbookingsbyhospitalid/{hospitalId}`
Actor	Any authenticated user

FR-BOOK-14 — Get All Bookings GET

Endpoint	`GET /api/vaxtrack/v1/booking/getallbookings`
Actor	Platform admin only

FR-BOOK-15 — Get Next Available Slot GET

Endpoint	`GET /api/vaxtrack/v1/booking/getnextavailableslot/{hospitalId}/{date}`
Actor	Any authenticated user

Behaviour: Read-only preview of the slot that would be assigned for a given hospital and date, shown to the user before they submit a booking.

FR-BOOK-16 — Is Booking Exists GET

Endpoint	`GET /api/vaxtrack/v1/booking/isbookingexists/{bookingId}`
Actor	Any authenticated user

FR-BOOK-17 — Delete Booking DELETE

Endpoint	`DELETE /api/vaxtrack/v1/booking/deletebooking/{bookingId}`
Actor	Platform admin only

Behaviour: Soft-deletes the booking and releases any dose still pending (neither completed nor already cancelled/rejected) back to hospital capacity.

FR-BOOK-18 — Get Certificate GET

Endpoint	`GET /api/vaxtrack/v1/booking/getcertificate/{bookingId}`
Actor	Anonymous (public)

Behaviour: Public, unauthenticated lookup by booking id — mirrors a real vaccination certificate's QR-code verification. Returns beneficiary name, age, gender, both hospitals, and completion dates; deliberately excludes email, phone, and address.

Business Rules: BR-22.

Error Cases: Booking not found or vaccination not yet complete → 404.

5.5 Module: Role & Access Requests

UserRoleMapping

Scoped role assignment and the hospital-admin self-service application queue

This module owns the `hospital-admin` scoped-role mapping table and the self-service application flow that lets a normal user apply for that role rather than requiring a platform admin to assign it unprompted. Role assignment is reactivation-safe: revoking and later re-granting the same role reuses the existing mapping row rather than creating a duplicate.

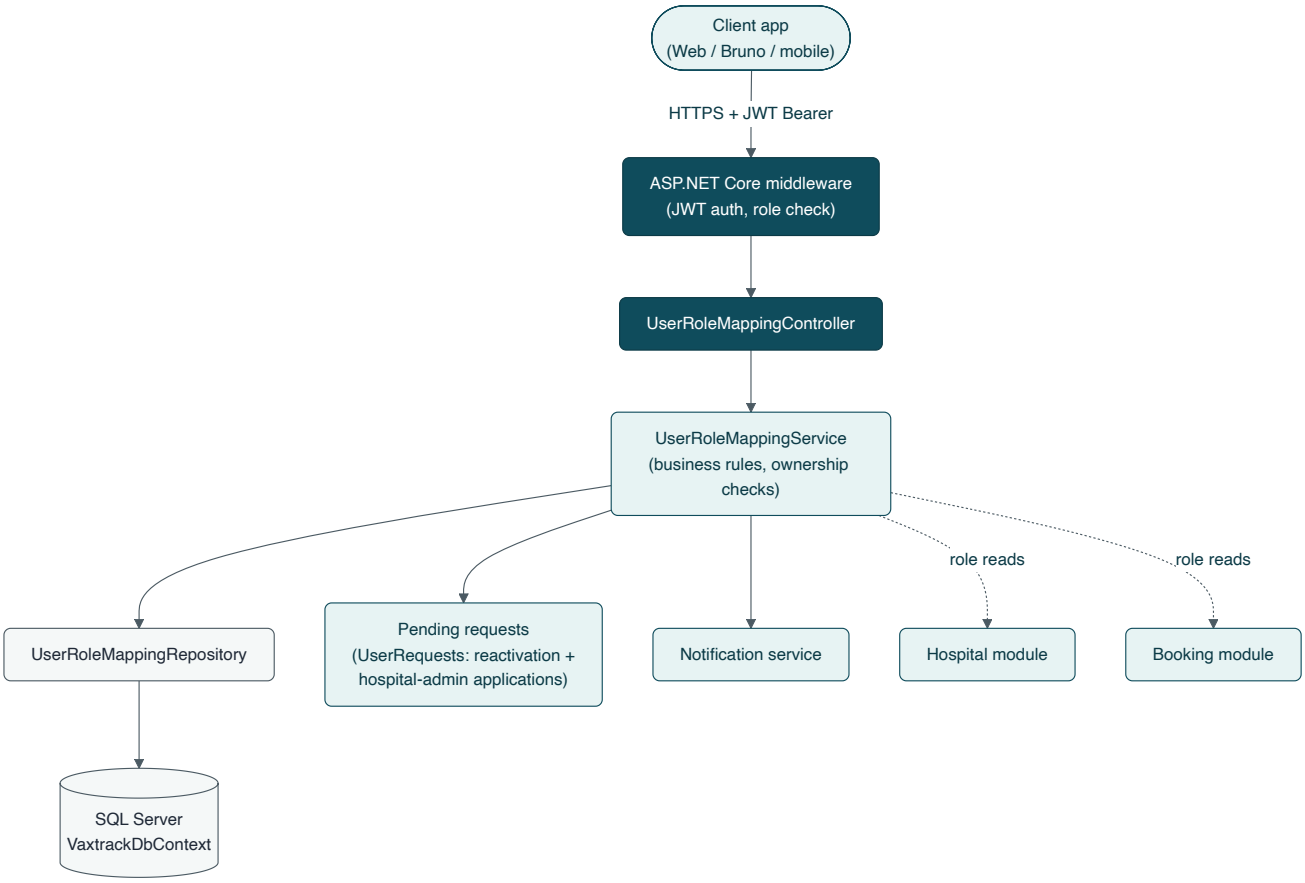

Figure 30. *High-Level Design — Role & Access Requests module.*

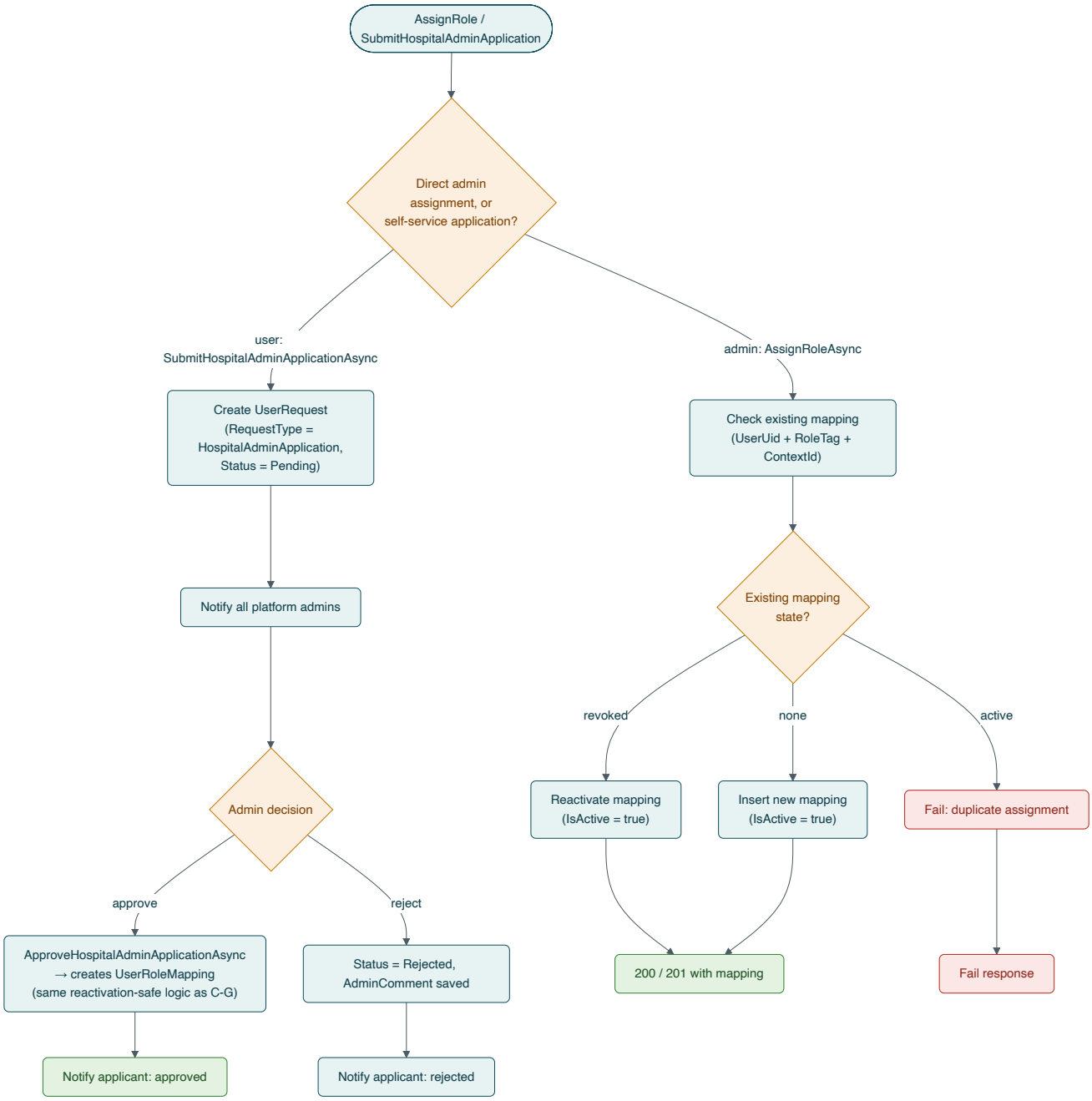

Figure 31. *Low-Level Design — direct assignment vs. self-service application routing.*

Key Flow

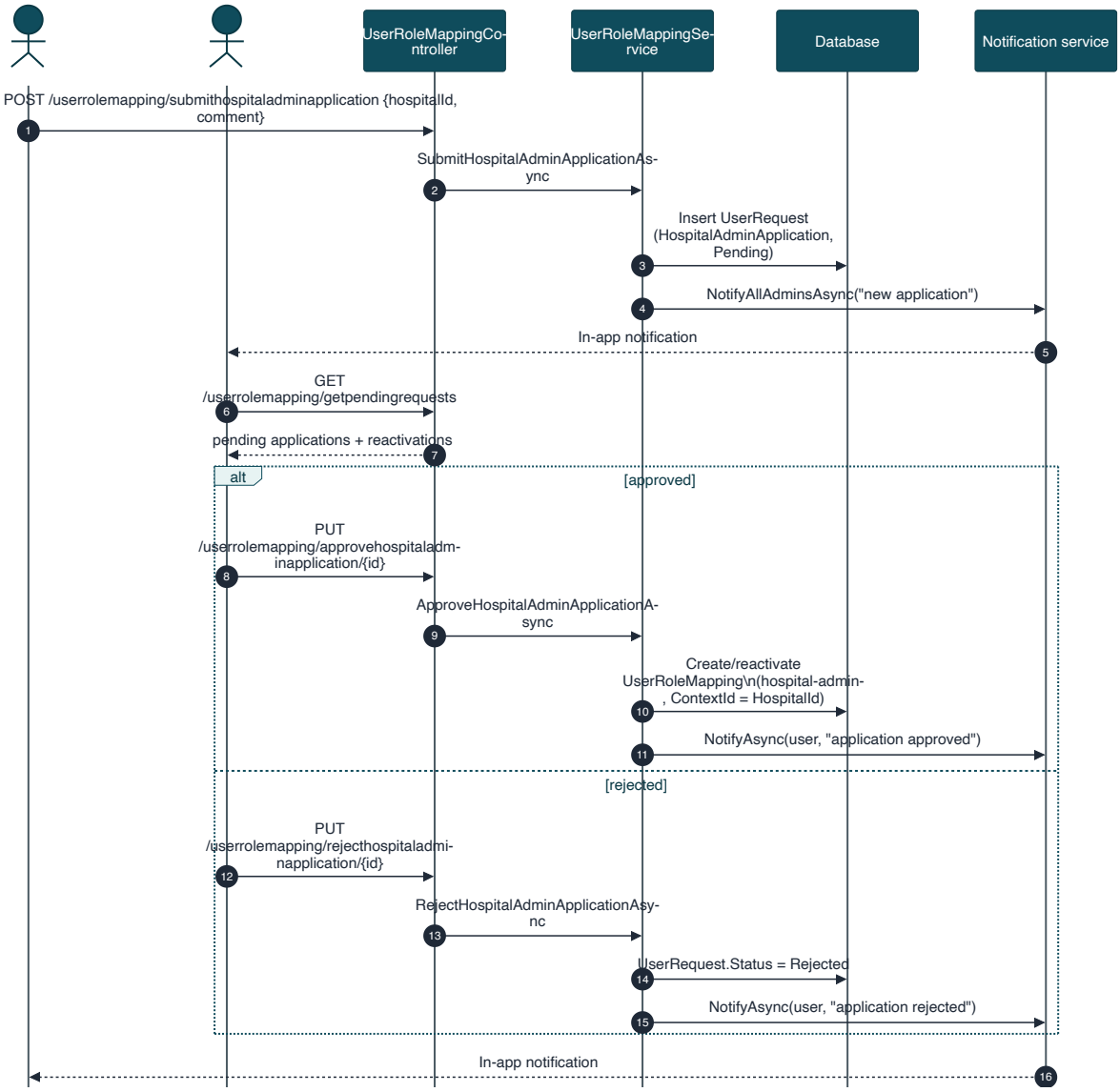

Figure 32. *Sequence — hospital-admin self-service application, approval, and rejection.*

Endpoint Requirements

FR-ROLE-01 — Assign Role — POST

Endpoint	`POST /api/vaxtrack/v1/userrolemapping/assignrole`
Actor	Platform admin only

Behaviour: Directly grants a scoped role. If an active mapping already exists, the request is rejected as a duplicate; if a revoked mapping exists, it is reactivated in place; otherwise a new mapping is created.

Business Rules: BR-10.

FR-ROLE-02 — Revoke Role — DELETE

Endpoint	`DELETE /api/vaxtrack/v1/userrolemapping/revokerole/{mappingId}`
Actor	Platform admin only

Behaviour: Soft-revokes the mapping. Effect is immediate — the very next request relying on that role fails its scope check.

FR-ROLE-03 — Get User Roles — GET

Endpoint	`GET /api/vaxtrack/v1/userrolemapping/getuserroles/{userUid}`
Actor	That user or platform admin

Behaviour: Lets a user discover their own scoped role assignments — necessary since scoped roles are never encoded in the JWT itself.

FR-ROLE-04 — Get Users In Role — GET

Endpoint	`GET /api/vaxtrack/v1/userrolemapping/getusersinrole/{roleTag}`
Actor	Platform admin only

Behaviour: Optionally narrowed to a specific context id (e.g. one hospital's admins); omitted, returns every user holding that role tag in any scope.

FR-ROLE-05 — Is User In Role — GET

Endpoint	`GET /api/vaxtrack/v1/userrolemapping/isuserinrole/{userUid}/{roleTag}`
Actor	That user or platform admin

FR-ROLE-06 — Submit Hospital-Admin Application — POST

Endpoint	`POST /api/vaxtrack/v1/userrolemapping/submithospitaladminapplication`
Actor	Any authenticated user

Behaviour: Raises a pending request naming the target hospital and an optional comment, and notifies every platform admin.

FR-ROLE-07 — Approve / Reject Hospital-Admin Application **PUT**

Endpoint	`PUT /.../approvehospitaladminapplication/{requestId}` | `PUT /.../rejecthospitaladminapplication/{requestId}`
Actor	Platform admin only

Behaviour: Approval creates (or reactivates) the underlying `hospital-admin` mapping for the named hospital using the same reactivation-safe logic as FR-ROLE-01; rejection simply closes the request with a comment. Either way, the applicant is notified.

Business Rules: BR-10.

FR-ROLE-08 — Get Pending Requests **GET**

Endpoint	`GET /api/vaxtrack/v1/userrolemapping/getpendingrequests`
Actor	Platform admin only

Behaviour: Returns every pending self-service request — both hospital-admin applications and account reactivation requests (Section 5.1, FR-AUTH-05) — as a single review queue.

5.6 Module: Notifications

Notification

The in-app alert layer that surfaces every lifecycle decision back to the affected user

This module is deliberately thin and purely additive: every other module calls into it, fire-and-forget, whenever something happens that the affected user or platform admins should know about. A notification failure never fails the action that triggered it — it is written alongside, not inside, the underlying transaction (BR-23).

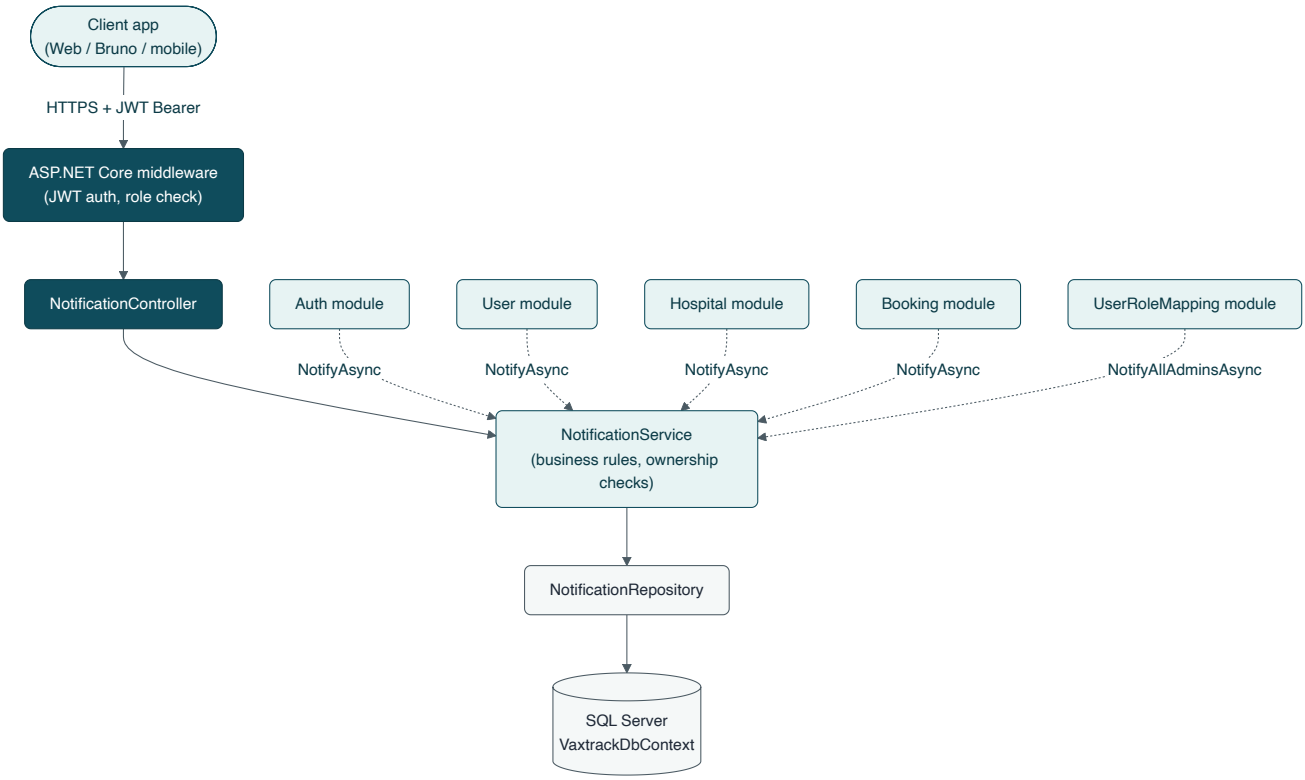

Figure 33. *High-Level Design — Notification module and its callers.*

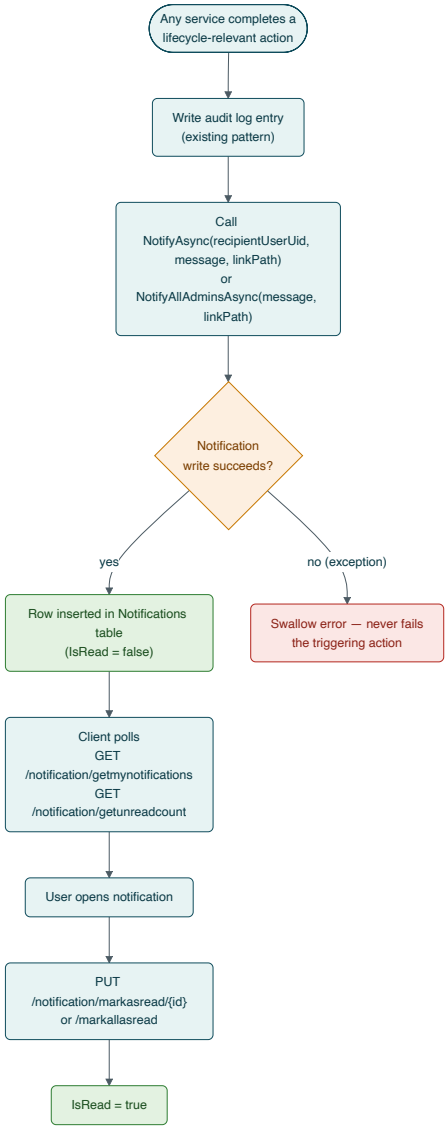

Figure 34. *Low-Level Design — fire-and-forget delivery and read-state flow.*

Key Flow

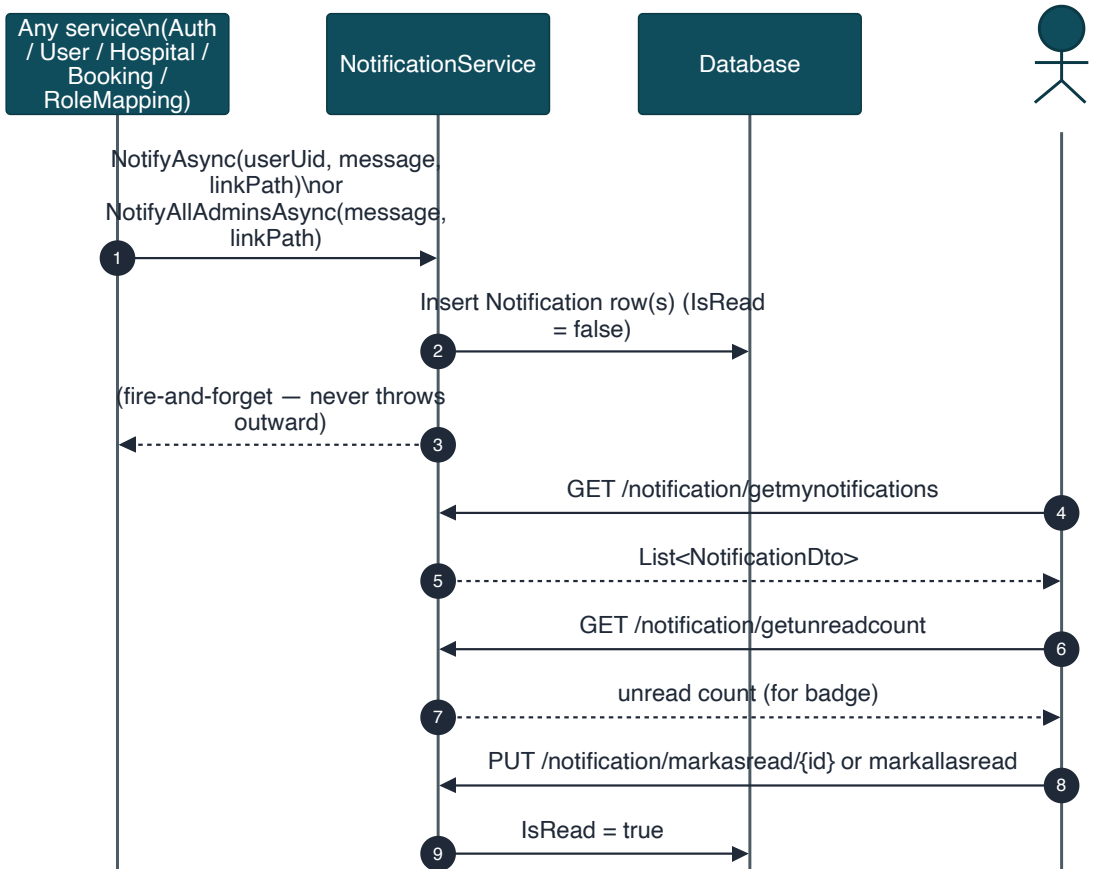

Figure 35. *Sequence — notification creation and consumption.*

Endpoint Requirements

FR-NOTIF-01 — Get My Notifications — GET

Endpoint	`GET /api/vaxtrack/v1/notification/getmynotifications`
Actor	Any authenticated user (self only)

FR-NOTIF-02 — Get Unread Count — GET

Endpoint	`GET /api/vaxtrack/v1/notification/getunreadcount`
Actor	Any authenticated user (self only)

Behaviour: Backs the unread-count badge in the frontend navigation bar.

FR-NOTIF-03 — Mark As Read	PUT
Endpoint	`PUT /api/vaxtrack/v1/notification/markasread/{id}`
Actor	Owner of the notification

FR-NOTIF-04 — Mark All As Read	PUT
Endpoint	`PUT /api/vaxtrack/v1/notification/markallasread`
Actor	Any authenticated user (self only)

◎ 6. Non-Functional Requirements

Category	Requirement
Security	All credentials are transmitted over HTTPS. Passwords are BCrypt-hashed, never stored or logged in plain text. JWTs are short-lived and are server-side revocable at logout. Every authorization decision (platform role and scoped role) is enforced server-side — the frontend never gates access on its own.
Auditability	Every state-changing action on a booking, hospital, or user is written to a permanent, append-only audit log capturing actor, action, timestamp, and (where applicable) a reason.
Data Retention	Deletion is soft throughout the platform — users, hospitals, bookings, and credentials are flagged inactive/removed, never physically erased, preserving full history for audit and dispute resolution.
Availability of Capacity Data	Hospital slot availability is updated synchronously and atomically with the booking action that consumes or releases it — no eventual-consistency window where capacity could be oversold.
Usability	Every account-lockout scenario (disabled account, forgotten password) has a documented, self-service recovery path that does not require manual database intervention.
Extensibility	Scoped roles are modelled generically (role tag + context id), so a future role beyond `hospital-admin` can be introduced without a schema change.
Observability	All service-layer errors are logged with the originating method name before being surfaced to the caller as a generic error, so failures are diagnosable without leaking internal detail to the client.

7. Business Rules

The rules below are cross-referenced by ID from the endpoint requirement cards in Section 5. They are the authoritative, consolidated statement of platform policy — where a business rule and a narrower FR description appear to differ, this section governs.

BR-01	A user cannot create a booking on behalf of another user — the authenticated caller's identity must match the booking's owner.
BR-02	A user may hold at most one active booking at a time.
BR-03	Dose 1 must be approved before Dose 2 can be booked.
BR-04	Dose 2 may be booked at a different hospital than Dose 1.
BR-05	A dose that has already been administered (completed) can never be cancelled or rejected.
BR-06	Cancelling or rejecting a pending dose restores exactly one slot to the relevant hospital.
BR-07	A hospital-admin may only approve or reject bookings at the hospital(s) they are scoped to.
BR-08	A hospital's available slot count is always kept within the range [0, TotalSlots]; a hospital-admin may only update the hospital(s) they are scoped to.
BR-09	Only a platform admin may perform a full-field administrative override of a booking, including marking a dose completed outside the normal approval flow.
BR-10	Role assignment is reactivation-safe — granting a previously revoked role reuses the existing mapping row rather than creating a duplicate active row.
BR-11	Permanently removing a hospital requires two-party confirmation: a platform admin initiates the unregister request, and the hospital's own hospital-admin (or the platform admin, if none is assigned) must separately authorize it with their own password.
BR-12	Deleting a user, whether self-initiated or admin-initiated, cascades to soft-delete their active bookings (releasing slots), revoke their role mappings, and soft-delete their credentials.
BR-13	A requested dose date/time must be in the future at the time of booking.
BR-14	An email address must be unique across all active (non-deleted) accounts.
BR-15	Changing a user's own email or password requires their current password, unless the caller is a platform admin.
BR-16	A user's birthdate must be in the past at registration; age is computed once at registration and is not recalculated afterward.

BR-17	A hospital-admin scoped role is always tied to exactly one hospital per mapping row; a user may hold the role for multiple hospitals via multiple independent rows.
BR-18	A platform admin unconditionally satisfies any scoped-role check without a database lookup.
BR-19	Authentication failure responses (wrong password, unknown email, or a disabled/deleted account) are identical in shape and message, to prevent account enumeration.
BR-20	A disabled account cannot authenticate under any circumstance until a platform admin approves its reactivation.
BR-21	A password-reset token is single-use and expires 15 minutes after issue.
BR-22	A vaccination certificate exposes only beneficiary name, age, gender, hospitals, and completion dates — never contact details such as email, phone, or address.
BR-23	A notification failure must never cause the triggering business action to fail; notification delivery is fire-and-forget.
BR-24	Passwords are always stored as a BCrypt hash; plain text is never persisted.
BR-25	Logging out revokes the current JWT server-side; a revoked token is rejected on every subsequent request even if it has not yet naturally expired.
BR-26	Disabling a user account or a hospital requires the platform admin to supply a reason; the reason is retained in the audit trail.

⚠ 8. Error Handling & Exception Scenarios

The platform uses a small, consistent set of outcomes across every endpoint. Business-rule violations and not-found conditions are reported as failures with a descriptive message; authorization failures are reported distinctly from validation failures so a client can tell "you're not allowed" apart from "that request doesn't make sense."

HTTP Status	Meaning	Typical Trigger
200 / 201 / 204	Success	Request completed; 201 for creation (with a Location header to the new resource), 204 for a successful action with no response body (e.g. delete, mark-as-read).
400 Bad Request	Validation failure	A supplied value fails a business-rule check the caller could reasonably fix — e.g. wrong password on Authorize Unregister, an invalid reason field.
403 Forbidden	Authorization failure	The caller is authenticated but does not own the target resource and does not hold the required platform or scoped role. Raised whenever a service-layer ownership or scope check fails.
404 Not Found	Resource state failure	Used narrowly — e.g. a certificate request for a booking that does not exist or is not yet vaccination-complete.
401 Unauthorized	Authentication failure	Missing, malformed, expired, or revoked JWT. Rejected by the authentication pipeline before any controller runs.
500 Internal Server Error	Generic / unexpected failure	Used deliberately for login and credential-related failures (BR-19, enumeration prevention) and as the catch-all for any unhandled service exception, which is always logged server-side first.

> ⚠ **Why login failures return 500, not 401**
>
> Returning a distinct status or message for "wrong password" versus "unknown email" versus "account disabled" would let an attacker enumerate registered emails or probe account status. VaxTrack deliberately collapses all three into the same generic failure at login (BR-19) — the one narrow exception is a disabled account, which surfaces a 403 with a machine-readable `ACCOUNT_DISABLED` code, but only after the password has already been verified, so a wrong-password guess cannot be used to fish for disabled accounts.

Cross-Cutting Exception Semantics

Exception	Resulting Response
`UnauthorizedAccessException` (service layer)	Caught by the controller and converted to 403 Forbidden.
`AccountDisabledException`	Caught by AuthController and converted to 403 with `{ code: "ACCOUNT_DISABLED", message, reason }`.
`ArgumentException`	Caught and converted to 400 Bad Request with the validation message.
Any other unhandled exception	Logged with the originating method name, then converted to a generic 500 response — internal detail is never returned to the client.

ⓘ 9. Assumptions & Constraints

9.1 Assumptions

- No outbound email service is integrated yet. Password-reset tokens are currently returned directly in the API response for development and test purposes; wiring up the stubbed email service to deliver them (and stripping them from the response) is a follow-on, not a redesign.
- A hospital's Dose 1 and Dose 2 slot number is caller-supplied; the platform does not perform server-side scheduling optimisation beyond enforcing capacity bounds.
- The platform does not enforce a minimum or maximum gap between a beneficiary's Dose 1 and Dose 2 dates.
- Profile picture storage is a path/URL string on the user record; the underlying file storage and CDN serving strategy is an infrastructure concern outside this specification.

9.2 Constraints

- Every authorization decision must be enforceable server-side without trusting any client-supplied role or identity claim beyond the validated JWT.
- No record may be physically deleted by the API — the soft-delete pattern (Section 6) applies uniformly across users, hospitals, bookings, and credentials.
- A hospital's available slot count must never fall outside [0, TotalSlots] under any sequence of concurrent booking operations.

⊗ 10. Out of Scope

The following are explicitly not addressed by the current release of VaxTrack, and are not implied by any requirement in Section 5:

- Outbound email or SMS delivery of notifications, password-reset tokens, or certificates (the Notification module is in-app only; email is stubbed — see Section 9).
- Payment processing of any kind — the platform does not charge for bookings.
- Third-party identity provider (SSO/OAuth) login — authentication is first-party email/password only.
- Multi-language / localisation support.
- Native mobile applications (the API is client-agnostic and could support one, but none is delivered as part of this scope).
- Automated appointment scheduling/optimisation across hospitals (slot selection is manual, caller-driven).
- Hard deletion or GDPR-style permanent erasure of personal data (the platform uses soft-delete uniformly; a hard-erasure capability would need separate legal and technical review).

11. Glossary

Term	Meaning
Active Booking	A booking whose vaccination is not yet fully completed and has not been fully cancelled/rejected — i.e. it still has an actionable dose.
Context Id	The value that narrows a scoped role to a specific entity — for `hospital-admin`, this is the target Hospital Id.
Dose Display Status	The server-computed, human-facing status of a dose (Pending / Completed / Cancelled / Rejected / Not Booked) derived from the underlying raw flags, so client screens never re-derive it themselves.
Reactivation-Safe	A write pattern where re-granting something previously revoked reuses the existing record instead of creating a duplicate.
Role Tag	The string identifier of a scoped role, e.g. `hospital-admin`.
Scope Check	A runtime lookup confirming a caller's scoped role applies to the specific entity being acted on.
Two-Party Confirmation	A safety pattern requiring two independent actors to agree before an irreversible action (hospital unregister) proceeds.
UserRequest	The generic pending-request record backing both account-reactivation requests and hospital-admin applications.

12. Appendix — Diagram Index

Figure	Title	Location
1	End-to-end functional flow	Section 3.3
2–5	Use case diagrams (Account & Access, Booking & Vaccination, Hospital Management, Platform Administration)	Section 4.4
6–7	Auth module — HLD, LLD	Section 5.1
8–11	Auth module — sequence diagrams (registration, login, forgot/reset password, account reactivation)	Section 5.1
12–13	User module — HLD, LLD	Section 5.2
14	User module — sequence diagram (profile self-service)	Section 5.2
15	User module — state diagram (account lifecycle)	Section 5.2
16–17	Hospital module — HLD, LLD	Section 5.3
18–20	Hospital module — sequence diagrams (create/update, disable/reactivate, unregister)	Section 5.3
21	Hospital module — state diagram (operational status lifecycle)	Section 5.3
22–23	Booking module — HLD, LLD	Section 5.4
24–27	Booking module — sequence diagrams (create/approve, Dose 2/completion, cancel/reject/edit/rebook, certificate)	Section 5.4
28–29	Booking module — state diagrams (Dose 1, Dose 2 lifecycles)	Section 5.4
30–31	Role & Access Requests module — HLD, LLD	Section 5.5
32	Role & Access Requests module — sequence diagram (hospital-admin application)	Section 5.5
33–34	Notification module — HLD, LLD	Section 5.6
35	Notification module — sequence diagram	Section 5.6

13. Approval & Sign-off

This Functional Specification Document represents the complete, current business and functional behaviour of the VaxTrack v2.0.0 platform. It is submitted for stakeholder review with the intent that, once approved, it becomes the agreed baseline against which the delivered system — and any subsequent change request — is measured.

Reviewers are asked to confirm, specifically: that the business objectives (Section 2.2) and success criteria (Section 2.4) are correctly captured; that the roles and permissions matrix (Section 4.3) matches the intended access model; that no required capability is missing from Section 5; and that the business rules (Section 7) reflect actual policy rather than incidental implementation behaviour.

Next Steps

1. Circulate to the distribution list in Document Control for review. 2. Hold a walkthrough / approval call to work through open questions section by section. 3. Record any agreed changes as a new revision in the Document Control history. 4. Upon sign-off below, this version becomes the baseline for the companion Architecture, Technical Specification, and Test Cases documents.

NAME	ROLE	SIGNATURE	DATE	DECISION
	Project Sponsor / Business Stakeholder			□ Approved □ Changes Requested
	Product / Delivery Lead			□ Approved □ Changes Requested
	Engineering Lead			□ Approved □ Changes Requested
	QA / Test Lead			□ Approved □ Changes Requested

VaxTrack — Functional Specification Document, Version 2.0.0, 08 July 2026. Confidential — prepared for internal stakeholder review.